

**SmartSplit:
Intelligent Bill Splitting & Expense Sharing
Application**

By
Prakash Bohara

Submitted to
The University of Roehampton

In partial fulfilment of the requirements
for the degree of
**BACHELOR OF SCIENCE in School of Arts, Humanities and Social
Science**

Declaration

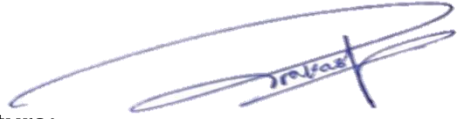
I, Prakash Bohara, hereby declare that this report is entirely my own work, submitted in partial fulfilment of the requirements for the degree of BSc (Hons) Computer Science at the University of Roehampton.

Where the work of others has been used, it has been clearly acknowledged through appropriate referencing and the use of quotation marks where necessary. All sources are listed in the references section.

This work is original and has not been previously submitted, in whole or in part, for the award of any degree at this or any other institution.

Name: Prakash Bohara

Date: May 2026

Signature:  _____

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Kimia Aksir, for their invaluable guidance, support, and expertise throughout this project. Their feedback and encouragement were instrumental in shaping both the direction and quality of this work.

I also wish to thank the University of Roehampton for providing the resources and facilities necessary to complete this project. I am grateful to my friends and family for their participation in user testing and for offering valuable feedback during the development process.

Finally, I acknowledge the contributions of the open-source community, whose libraries and frameworks made this project possible, particularly the teams behind Flutter, Firebase, and Google ML Kit.

Abstract

Managing shared expenses among students, flatmates, and travel groups remains an everyday source of friction. Manual data entry is tedious, settlement frequently requires more transactions than necessary, and existing applications either restrict useful features behind paywalls or offer no intelligent automation. SmartSplit addresses these gaps by combining on-device receipt recognition, real-time multi-device synchronisation, and a provably optimal settlement algorithm in a single, freely available Android application.

The application is built in Flutter, with Firebase providing authentication, real-time data synchronisation, server-side logic, and tester distribution. Optical character recognition is performed on-device through Google ML Kit, supporting offline operation and protecting user privacy. The settlement component is implemented as a Minimum-Cost Maximum-Flow (MCMF) optimisation that minimises the number of transactions required to clear group debts, with optimality guaranteed by the max-flow min-cut theorem. A defence-in-depth security model combines salted SHA-256 OTP authentication with atomic-transaction rate limiting, Firestore rules gated by email verification, AES-256-CBC encrypted bank details, and biometric application lock.

Functional, security, and pilot user-acceptance testing returned a mean rating of 7.71 out of 10 and a 71.4% real-life-use intent across seven respondents. These are indicative results from a small sample, but consistent across multiple independent measures. The evaluation also exposed limitations: OCR accuracy degrades on non-standard receipts, the user interface supports selective equal-split with pay-on-behalf but not percentage or exact split modes, and automated test coverage is concentrated on the settlement algorithm. These limitations are reported honestly and identified as concrete future work in Chapter 9.

Table of Contents

1	Introduction	9
1.1	Motivation.....	9
1.2	Problem Statement and Research Question	9
1.3	Aims	10
1.4	Objectives	10
2	Background and Context	12
2.1	Background	12
2.2	Literature Review.....	12
2.3	Competitor Analysis	15
3	Tools, Methods, and Project Planning.....	17
3.1	Tools and Frameworks.....	17
3.2	Methods	18
3.3	Project Management and Product Lifecycle.....	19
3.4	Legal, Ethical and Professional Issues	21
3.5	Risk Register	22
4	System Requirements and Specifications.....	24
4.1	User Personas.....	24
4.2	User Stories	25
4.3	Wireframes	26
4.4	User Flow Diagram	28
4.5	Summary	30
5	Solution Design	31
5.1	System Architecture	31
5.2	Domain Model.....	32
5.3	Use Cases	33
5.4	Database Design	34
5.5	Key Workflow Sequences	36
5.6	Algorithm Design	38
5.7	Summary	41
6	Solution Implementation	42
6.1	Development Environment.....	42
6.2	Sprint 1: Authentication and Account Security.....	42
6.3	Sprint 2: Group Management.....	44

6.4	Sprint 3: Expense Management with OCR.....	45
6.5	Sprint 4: Settlement Optimisation and Bank Details	47
6.6	Sprint 5: Notifications	48
6.7	Sprint 6: Analytics, Settings, and Profile	50
6.8	Cross-cutting Concerns	52
6.9	Implementation Summary	52
7	Solution testing and Evaluation	54
7.1	Testing Strategy	54
7.2	Unit Testing.....	54
7.3	Functional Testing	56
7.4	Security Testing	57
7.5	User Acceptance Testing	58
7.6	Strengths, Weaknesses, and Limitations.....	61
8	Discussion	63
8.1	Outcomes Against Project Aims	63
8.2	What the User Feedback Revealed	63
8.3	Engineering Lessons Learned	64
9	Conclusion.....	66
9.1	Summary	66
9.2	Recommendations and Future Work.....	66
9.3	Reflection	67
10	References	69
	Appendix A: Ethics Approval and Participant Consent.....	1
	Appendix B: User Personas	1
	Appendix C: User Journeys.....	5
	Appendix D: UAT Form and Sample Responses	6
	Appendix E: Project Proposal Poster	10

List of Figures

<i>Figure 3.1: Gantt chart showing project phases and assessed milestones across the academic year</i>	<i>20</i>
<i>Figure 3.2: new version released in app distribution</i>	<i>21</i>
<i>Figure 4.1: Group Creation Figma wireframe</i>	<i>27</i>

Figure 4.2: Home Screen Figma wireframe	27
Figure 4.3: Settlement View Figma wireframe	28
Figure 4.4: Analytics Dashboard Figma wireframe.....	28
Figure 4.5: Expense Entry Figma wireframe	28
Figure 4.6: Primary user flow through SmartSplit's core workflows	29
Figure 5.1: Three-tier system architecture: Flutter UI, services and providers, and the Firebase backend with on-device ML Kit	32
Figure 5.2: Class diagram showing the seven domain models and their relationships..	33
Figure 5.3: Use case diagram showing four actors and five functional packages	34
Figure 5.4: Firestore document model showing collections, subcollections, and embedded structures	35
Figure 5.5: OCR Expense Addition — sequence diagram	37
Figure 5.6: Settlement Calculation — sequence diagram	38
Figure 5.7: Debt settlement optimisation using minimum transactions.....	40
Figure 6.1: Authentication screens — registration with strength indicator, OTP entry, and email verification.....	44
Figure 6.2: Group creation screen and group details screen showing member list and balances	45
Figure 6.3: Scan Receipt screen with extracted data preview and confirmation	46
Figure 6.4: Settlement plan screen showing recommended payments and bank-detail expansion	48
Figure 6.5: Notification centre with badge counter and notification preferences screen	49
Figure 6.6: profile screen, Settings screen, password biometric and security screen ...	51
Figure 6.7: Analytics dashboard with category breakdown chart, monthly trend and top contributor.....	51
Figure 7.1: Test runner output showing all 9 tests passing	55
Figure 7.2: Firebase App Distribution dashboard showing release v1.0.1 with 9 invited / 7 accepted / 7 downloaded	58
Figure 7.3: Google Form summary — overall rating distribution and would-use-in-real- life chart	60

List of Tables

Table 2.1: Feature comparison across leading expense-sharing applications	15
Table 3.1: Primary tools and frameworks.....	17
Table 3.2: Project risk register (summary).....	22
Table 4.1: Persona summary.....	24
Table 5.1: Firestore collections.....	35
Table 6.1: Sprint summary.....	52
Table 7.1: Settlement algorithm test suite	55

<i>Table 7.2: Functional testing results, mapped to requirements</i>	<i>56</i>
<i>Table 7.3: Headline (User Acceptance Test, n = 7) result</i>	<i>59</i>
<i>Table 7.4: records the changes.....</i>	<i>60</i>

1 Introduction

1.1 Motivation

Shared expenses are a routine feature of social and professional life among students, shared households, work teams, and travel groups yet the tools available to manage them remain surprisingly limited. Manual records, spreadsheets, and informal messages are still common, and each is prone to error, duplication, and dispute. The cognitive load of tracking who paid for what, who owes whom, and how to settle most efficiently falls disproportionately on the most organised member of any group, and frequently delays repayment for weeks.

A range of mobile applications addresses this problem space, with Splitwise [28], Tricount [30], and Settle Up [29] among the most widely used. Despite their popularity, all share recurring weaknesses: most do not extract data from receipts and require manual entry; many lack genuine real-time synchronisation across devices; their settlement procedures often suggest more transactions than mathematically necessary; and several restrict their most useful features to paid tiers. SmartSplit is proposed as an alternative that addresses each of these gaps explicitly. It is an Android application built with Flutter and backed by Firebase, with on-device receipt recognition supplied by Google ML Kit [10] and a settlement engine implemented as a Minimum-Cost Maximum-Flow optimisation. This report documents its design, implementation, and evaluation.

1.2 Problem Statement and Research Question

Existing expense-sharing applications fall short on four dimensions that a single integrated solution could address. First, intelligent automation is rare: receipt scanning is restricted to paid tiers in market-leading applications, and the recognition that does exist is fragile on non-standard layouts. Second, real-time synchronisation across devices is inconsistently implemented, with several applications relying on periodic polling or manual refresh. Third, settlement procedures frequently suggest more transactions

than the theoretical minimum, increasing friction in real settlements. Fourth, sensitive financial data, bank details, payment references, transaction history is rarely encrypted at rest in any defensible way. A comparative analysis of six leading applications (Section 2.3) demonstrates that no single existing solution closes all four gaps simultaneously.

The research question that follows is therefore:

How can a mobile application combine on-device machine learning, real-time cloud synchronisation, and an optimal settlement algorithm to improve the accuracy and efficiency of shared expense management, while protecting user data?

1.3 Aims

The primary aim of the project is to design, build, and evaluate an intelligent expense-sharing mobile application that addresses the four gaps identified above through automation, on-device machine learning, and user-centred design. Four specific aims operationalise this:

1. **Automate bill-splitting** through machine-learning-based receipt scanning, eliminating manual data entry and calculation errors for the most common case.
2. **Minimise the number of transactions** required to settle group debts by implementing an optimal settlement algorithm.
3. **Support real-time multi-device collaboration** through cloud-based synchronisation of shared group state.
4. **Enhance financial awareness** through analytics that present spending patterns by category, period, and member contribution.

1.4 Objectives

The following objectives operationalise the aims above and structure the remainder of the report:

1. Analyse existing expense-sharing applications and identify their functional, technical, and usability limitations (*supports Aim 1; Chapter 2*).
2. Design a scalable system architecture incorporating user authentication, real-time data synchronisation, and secure cloud storage (*supports Aims 1 and 3; Chapter 5*).
3. Develop a cross-platform mobile application using Flutter that supports group creation, expense tracking, and balance calculation (*supports Aim 1; Chapter 6*).
4. Implement on-device receipt recognition using Google ML Kit to reduce manual entry and to support offline use (*supports Aim 1; Chapter 6*).
5. Apply an optimal settlement algorithm to minimise the number of transactions required to resolve group debts (*supports Aim 2; Chapters 5 and 6*).
6. Implement an analytics dashboard providing per-category, per-period, and per-member spending insights (*supports Aim 4; Chapter 6*).
7. Evaluate the system through unit, functional, security, and user-acceptance testing to assess accuracy, performance, and usability (*supports all aims; Chapter 7*).

Each subsequent chapter advances one or more of these objectives, and the discussion in Chapter 8 returns to assess outcomes against the aims.

2 Background and Context

2.1 Background

The digitalisation of personal finance has moved from desktop spreadsheets in the 1990s to dedicated mobile applications with the widespread adoption of smartphones in the late 2000s. The financial-technology sector has since grown into a distinct industry [1], with bill-splitting and expense-management applications emerging as a recognisable category serving households, travel groups, and roommate cohorts.

Despite this proliferation, three persistent weaknesses limit how useful these applications are in practice. First, manual data entry remains the dominant workflow, even though optical character recognition (OCR) is mature enough to extract structured data from receipts with reasonable accuracy [9]. Second, settlement procedures rarely guarantee the minimum number of transactions, so groups often end up with more payments to coordinate than is mathematically necessary [19]. Third, sensitive financial data particularly bank account details and payment references is rarely encrypted at rest in any application-level way; clients depend entirely on the underlying cloud provider's at-rest encryption.

These weaknesses motivate the four design priorities of SmartSplit (Section 1.2) and frame the literature review that follows.

2.2 Literature Review

This section reviews the literature relevant to SmartSplit across four themes that correspond to the project's main technical concerns: cross-platform mobile development, OCR for receipt processing, real-time backend infrastructure, and algorithmic approaches to debt settlement.

Cross-platform mobile development. The choice of mobile-development framework affects performance, productivity, and long-term maintainability. Cross-platform

frameworks have been increasingly favoured because they enable a single codebase to target multiple operating systems [2], reducing the maintenance overhead of separate native builds. Of the leading frameworks, Flutter [3] compiles to native code via the Dart toolchain, which avoids the runtime overhead introduced by JavaScript-bridge alternatives such as React Native [4]. This compilation strategy matters most when the user interface is animation-heavy or when scrolling performance is critical, both of which apply to a list-driven expense-management application. Backend-as-a-Service alternatives such as AWS Amplify [5] offer comparable client-tier capabilities but introduce configuration complexity disproportionate to a solo-developer project.

Optical character recognition. Optical character recognition has advanced substantially through deep learning, with convolutional and transformer-based models replacing the template-matching approaches of earlier systems [6]. Receipt processing is acknowledged as a particularly challenging OCR domain because of the variability in layout, merchant formatting, paper quality, and lighting conditions encountered in real use [8]. The ICDAR 2019 competition demonstrated that combining text recognition with downstream parsing, for example natural-language extraction of fields such as total, date, and merchant achieves substantially better accuracy on structured fields than raw recognition alone [9]. Two implementation options dominate the consumer mobile space: Google ML Kit provides on-device text recognition with no per-request cost, full offline capability, and a privacy guarantee that no image leaves the device [10]; cloud-based alternatives such as Google Vision API [11] offer higher general-purpose accuracy at the cost of network dependence, per-request charges, and the requirement to send user-captured images to a third-party endpoint. The Tesseract project [7] remains a viable open-source alternative for non-mobile deployments but typically requires more aggressive image preprocessing.

Real-time backend infrastructure. Mobile applications increasingly rely on Backend-as-a-Service platforms that bundle authentication, persistence, file storage, and notification delivery into a single managed product. Firebase, Google's BaaS platform, is well-suited to collaborative applications: Cloud Firestore [12] provides a real-time NoSQL document database with automatic synchronisation across connected clients,

supported by a document-based data model [13] that maps naturally onto the nested-group, nested-expense structure of an expense-sharing application. The platform additionally provides authentication primitives [14], a security-rules engine for declarative authorisation [15], and reference documentation that anchors many of the design decisions in Chapter 5 [16]. Bonneau et al. provide the canonical comparative framework for the relative strengths of password-based, token-based, and biometric authentication schemes that informs the layered approach taken here [17].

Algorithmic approaches to debt settlement. The debt-settlement problem can be formulated as graph optimisation: members are nodes, balances are weighted edges, and the objective is to settle all balances using the minimum number of transactions [18]. Verhoeff [19] showed that the general problem of finding the minimum number of transactions among a set of debts is NP-complete, which has two practical consequences. First, exact solutions become computationally infeasible for very large groups; second, heuristic algorithms are widely used in practice but cannot in general guarantee optimality.

Two strategies dominate the literature. Greedy heuristics [20] iteratively pair the largest creditor with the largest debtor; they are fast and produce reasonable results but offer no formal guarantee on transaction count. The alternative is to model settlement as a Minimum-Cost Maximum-Flow (MCMF) problem on a network where each potential debtor-to-creditor edge carries unit cost [21]. Under this formulation, the minimum-cost flow is also the minimum-transaction settlement, with optimality guaranteed by the max-flow min-cut theorem. The cost is greater computational complexity, but for the small group sizes typical of consumer expense-sharing the computation remains well within interactive response time. SmartSplit therefore adopts MCMF, accepting the additional complexity for the optimality guarantee.

FinTech adoption and user behaviour. Beyond the strictly technical literature, two studies of FinTech adoption inform the design priorities. Lee and Shin [22] identify three factors as consistently dominant in user adoption decisions: security and privacy, ease of use, and feature richness. Gomber et al. [23] complement this by showing that automation and intelligent features significantly improve user satisfaction and retention by

reducing the cognitive load associated with financial tracking. Together these findings motivate three of the four project aims (automated receipt scanning, real-time collaboration, and analytics) and justify the disproportionate effort spent on security primitives in Chapter 6.

2.3 Competitor Analysis

The expense-sharing market contains several established consumer applications. This section compares the six most prominent against SmartSplit to identify where existing solutions fall short.

A summary appears in Table 2.1, with each application's salient characteristics drawn from its public documentation and freely available app-store listing. The criteria reflect the four problem-statement gaps identified in Section 1.2.

Table 2.1: Feature comparison across leading expense-sharing applications

FEATURE	SPLITWISE	SETTLE UP	TRICOUNT	SPLID	VENMO	TAB	SMARTSPLIT
OCR RECEIPT SCANNING	Pro tier only	No	No	No	No	No	Yes ✓
REAL-TIME SYNC	Yes	Manual refresh	Yes	Polling	Yes	N/A	Yes ✓
SETTLEMENT OPTIMIZATION	Greedy	Greedy	Basic	None	None	N/A	MCMF (Probably optimal)
VISUAL ANALYTICS	Pro tier only	No	Limited	No	No	No	Yes ✓
OFFLINE OCR	No	No	No	No	No	No	Yes ✓
FREE ACCESS	Partial	Yes	Yes	Yes	Yes	Yes	Yes ✓
MULTI-PLATFORM	Yes	Yes	Yes	Yes	Yes	No	Yes (Android ✓ ; ios deferred)

Notes: "Pro tier only" indicates the feature is restricted to a paid subscription. "Greedy" indicates a heuristic settlement that does not guarantee minimum transactions.

"Polling" indicates periodic refresh rather than push-based updates. "N/A" indicates the feature is outside the application's scope.

Splitwise dominates the market but restricts OCR and analytics to its paid tier, conflicting with this project's open-access goal. Settle Up and Tricount provide acceptable settlement and group management but lack OCR entirely, leaving the largest source of user friction unaddressed. Splid relies on periodic polling rather than push synchronisation, producing visible delays between expense entry and balance updates on other devices. Venmo is primarily a peer-to-peer payment platform with expense splitting as an incidental feature, lacking settlement optimisation and analytics. Tab is restricted to restaurant bill splitting and does not support general expense tracking or multi-event group management.

No existing application combines all four capabilities: OCR receipt scanning, true real-time synchronisation, optimal settlement, and encrypted bank-detail storage in a single freely accessible product. SmartSplit closes this gap; the architecture and implementation chapters that follow describe how.

3 Tools, Methods, and Project Planning

This chapter describes the tools and methods used to develop SmartSplit, the project-management approach adopted, and the legal, ethical, and risk considerations that shaped the project.

3.1 Tools and Frameworks

The tooling decisions for the project balanced developer productivity against the need to keep the system architecture defensible for an academic submission. The primary technologies are summarised in Table 3.1 and discussed below.

Table 3.1: Primary tools and frameworks

Category	Tool	Role
Mobile framework	Flutter	Cross-platform UI rendered to native code
Backend	Firebase	Authentication, real-time database, server logic
OCR	Google ML Kit	On-device receipt text recognition
IDE	Visual Studio Code	Source editing
Design	Figma	Wireframing and visual design
Modelling	PlantUML	UML diagrams
Version control / project management	GitHub	Source control, Kanban board, Gantt timeline

Flutter was selected over React Native and AWS Amplify on three grounds. First, Flutter compiles directly to native code, avoiding the JavaScript bridge runtime overhead that affects scrolling and animation performance in alternatives [4]. Second, Flutter's widget composition model maps naturally onto reactive state management with the Provider package, supporting the layered architecture described in Chapter 5. Third, Dart's strong type system catches a class of errors at compile time that would otherwise emerge only at runtime, a meaningful productivity benefit on a solo project.

Firebase was selected as the backend platform over alternatives such as AWS Amplify [5] due to its real-time data synchronisation capabilities [12], which are a core feature rather than an additional service. In addition, Firebase's security rules engine [15] enforces access control declaratively at the database level, reducing reliance on application-layer logic. A more detailed justification of individual Firebase services including Authentication [14], Firestore [13], and Cloud Functions is provided in Chapter 5.

Google ML Kit [10] was chosen for receipt OCR because it performs text recognition entirely on-device. This approach removes network dependency, eliminates per-request costs, and ensures that user-captured receipt images remain local to the device. However, this introduces a trade-off: recognition accuracy may be lower for non-standard receipt formats compared to cloud-based solutions such as the Google Vision API [11]. This limitation is evaluated further in Chapter 7.

3.2 Methods

The project followed an iterative, user-centred approach rather than a formal framework such as Scrum. This decision reflects the constraints of a solo developer working over an academic year, where structured Scrum ceremonies (e.g., daily stand-ups, sprint reviews, and retrospectives) would introduce unnecessary overhead without delivering proportional value. However, the project adopted the core Agile principles outlined in the Manifesto for Agile Software Development [24], prioritising working software over comprehensive documentation, responding to change over rigid planning, and maintaining continuous collaboration with the supervisor as the primary stakeholder.

User-centred design (UCD) [25][26] informed the process of understanding user needs and validating design decisions. This followed an iterative cycle of identifying the context of use, defining user requirements, developing design solutions, and evaluating them against those requirements. Early-stage user research was conducted informally through discussions with prospective users to identify common pain points in existing expense-sharing tools.

Development progressed through iterative cycles, each focusing on a specific feature area, including authentication, group management, OCR-based expense capture, settlement optimisation, notifications, and analytics. Each cycle began with task definition using a GitHub Kanban board, incorporated supervisor feedback upon completion, and concluded with a brief retrospective to inform subsequent development priorities.

3.3 Project Management and Product Lifecycle

Work tracking. Project planning and tracking used a small, integrated toolset built around the project's GitHub repository. The repository is publicly accessible at github.com/boharap1/SmartSplit, and the commit history, pull requests, and issue threads provide an independently verifiable record of the development process described in Chapter 6.

Timeline and milestones. A Gantt chart (Figure 3.1) provided the high-level project schedule, mapping each phase across the academic year and explicitly marking the three assessed milestones: Proposal Poster (Nov 2025), Mid-Project Review (Feb 2026), and Final Submission (May 2026). The schedule allocated approximately equal time to research-and-design and to implementation, with shorter phases reserved for testing, evaluation, and report polish at the end. The Gantt was reviewed fortnightly with the supervisor to track actual progress against the planned schedule and to adjust scope when emerging risks (Section 3.5) demanded it.

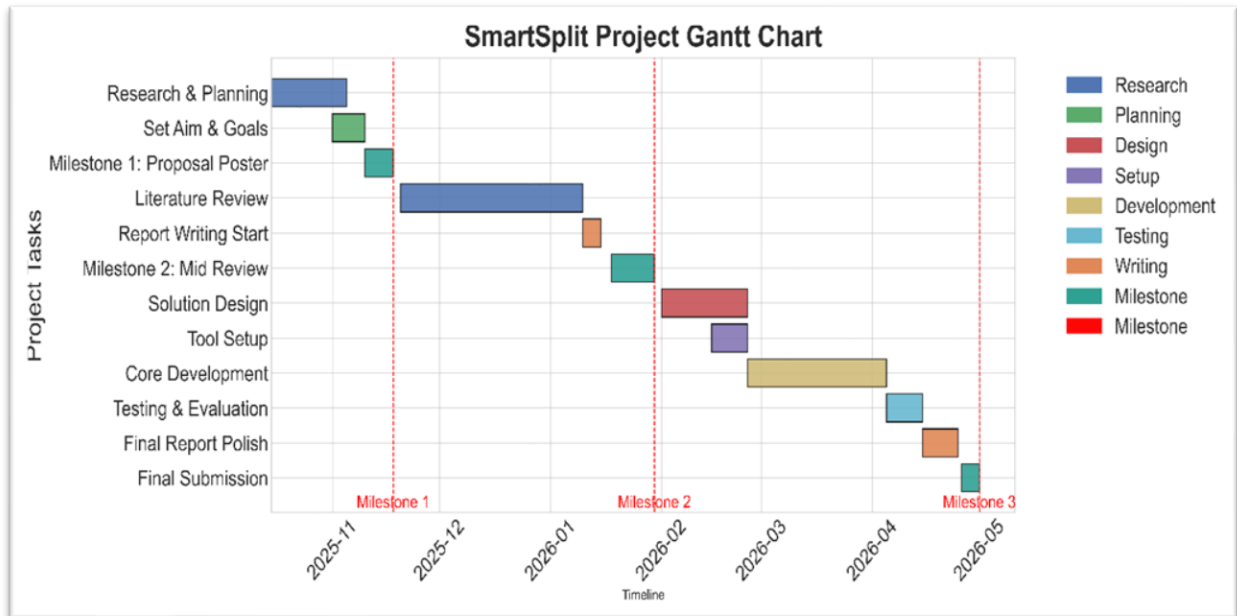


Figure 3.1: Gantt chart showing project phases and assessed milestones across the academic year

Iterative cycles within Core Development. Within the Core Development phase, work proceeded in iterative cycles, each focused on a specific feature area: authentication (Sprint 1), group management (Sprint 2), expense management with OCR (Sprint 3), settlement optimisation (Sprint 4), notifications (Sprint 5), and analytics with settings and profile management (Sprint 6). Each cycle began with task definition on the Kanban board, included supervisor review at completion, and concluded with a brief retrospective that informed the priorities of the next cycle.

Release management. Firebase App Distribution served as the release pipeline for tester builds during user-acceptance testing (Chapter 7). Each

`appDistributionUploadRelease` Gradle task uploaded a signed APK and notified registered testers automatically, supporting the v1.0.0 → v1.0.1 iterative-feedback cycle described in Section 7.5.

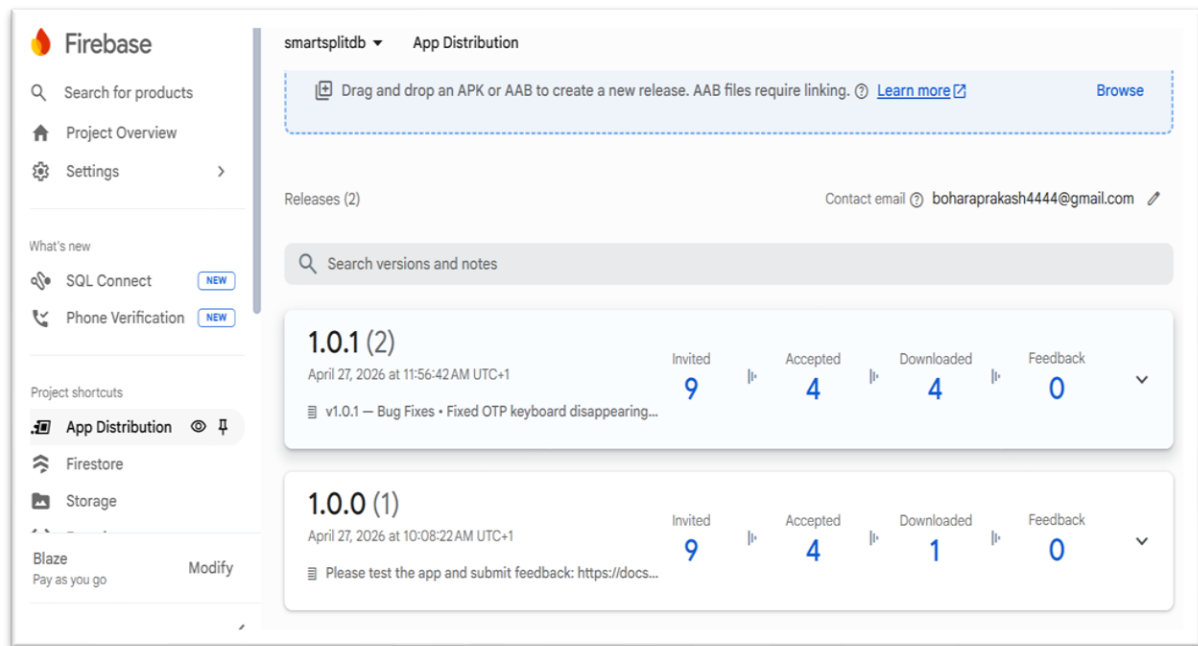


Figure 3.2: new version released in app distribution

3.4 Legal, Ethical and Professional Issues

GDPR compliance. SmartSplit applies data minimisation, purpose limitation, explicit user consent at registration, and account-deletion functionality for the right to erasure [38]. Password storage is delegated to Firebase Authentication [14], which applies industry-standard salted hashing internally; one-time passcodes are stored by Cloud Functions as salted SHA-256 hashes, as detailed in Section 6.3.

Ethical conduct. User research and pilot testing followed the ethical guidelines required by the University of Roehampton, with all participants providing informed consent (see Appendix A). Personal data was limited to email addresses required for App Distribution and Google Form responses.

Intellectual property. Source code is authored by the developer and submitted in accordance with University of Roehampton coursework regulations. Third-party libraries are used under their respective open-source licences with attribution.

Professional responsibility. Defensive engineering practices were applied throughout: encryption at rest for sensitive financial data, server-side validation for privileged operations, declarative authorisation through Firestore security rules, input validation on user-supplied data, structured error reporting through Crashlytics, and accessibility considerations in the user interface.

3.5 Risk Register

Table 3.2 below presents the key risks identified during planning, with their likelihood, impact, mitigation strategies, and current status.

Table 3.2: Project risk register (summary)

ID	Risk Description	Likelihood	Impact	Mitigation Strategy	Status
R1	OCR accuracy insufficient for practical use	Medium	High	Early proof-of-concept testing; manual entry fallback implemented	Mitigated
R2	Firebase costs exceed free-tier limits	Low	Medium	Monitor usage; optimise queries; remained within free-tier throughout	Monitored
R3	Time constraints prevent feature completion	Medium	Medium	MoSCoW prioritisation; focus on MVP; defer Could-Have features	Mitigated
R4	Flutter/Dart learning curve delays implementation	Medium	Medium	Complete tutorials before main implementation; allocate dedicated time	Mitigated
R5	Real-time synchronisation conflicts	Low	High	Apply Firestore conflict resolution; concurrent testing	Monitored
R6	Late discovery of usability issues	Medium	High	Early wireframe testing; iterative prototyping with prospective users	Mitigated
R7	Security vulnerabilities	Low	High	Follow Firebase best practices; implement security rules; security testing (Section 7.4)	Mitigated

R8	Loss of work due to hardware failure	Low	High	Daily GitHub commits; cloud backups of non-code assets	Monitored
R9	Algorithm performance degradation with very large groups	Medium	High	Real-world group sizes (typically <10 members) keep complexity bounded; large-scale stress testing identified as future work	Accepted
R10	Third-party service deprecation (Firebase, ML Kit)	Very Low	High	Monitor announcements; maintain modular service-layer abstractions for portability	Monitored
R11	Firestore security rule gaps after schema changes	Medium	High	Test all CRUD paths after rule changes; subcollection rule gap caught during integration and patched in v1.0.1 (Section 6.6)	Realised and resolved

Key mitigations. Three risks had a material impact on the project and informed key design decisions. R1 was mitigated through early proof-of-concept testing of ML Kit OCR, with a manual-entry fallback retained throughout development to ensure reliability. R3 was addressed using MoSCoW prioritisation, with lower-priority features (US11: currency support and US12: CSV export) explicitly deferred to future work. R11, the Firestore subcollection rules gap, was the only risk that materialised during development; it is documented here and revisited in the engineering lessons discussed in Chapter 8.

4 System Requirements and Specifications

This chapter presents the requirements for SmartSplit through user-centred documentation: personas summarising the target user base, MoSCoW-prioritised user stories, three workflow journeys, Figma wireframes, and a user flow diagram. full persona cards are provided in Appendix B.

4.1 User Personas

Three personas, representing the breadth of SmartSplit’s target users: a casual student, a security-conscious professional, and an occasional, travel-focused user were developed from the early-stage user research outlined in Section 3.2. Their characteristics, key challenges, and the user stories associated with each persona are summarised in Table 4.1, with detailed persona profiles provided in Appendix B. Each persona aligns with a distinct subset of user stories, ensuring that all requirements are grounded in clearly defined user needs.

Table 4.1: Persona summary

	Angel	Tripta	Dikshya
Role	University student	Young professional	Frequent traveller
Context	Shared housing; frequent low-value group expenses (groceries, bills, takeaways)	Workplace and social group expenses; weekend trips	International trips with rotating group composition
Primary pain points	Tracking who paid what; awkward debt conversations; no formal financial training	Coordination across distributed group members; security of bank details when settling	Multi-currency tracking; review of trip spending afterwards

Driving user stories	US1, US2, US3, US5, US7	US5, US6, US9, US10	US3, US8, US11
Persona card	(full card in Appendix B, Figure B.1)	(Appendix B, Figure B.2)	(Appendix B, Figure B.3)

4.2 User Stories

User stories capture functional requirements from the user perspective, prioritised using the MoSCoW method.

Must Have:

US1. As a user, I want to register an account with my email so that I can securely access my expense data across devices.

US2. As a group creator, I want to generate a shareable invitation code so that I can easily add members without requiring their contact details.

US3. As a group member, I want to scan receipt images with my phone camera so that expense details are automatically filled without manual typing.

US4. As a payer, I want to specify how an expense should be split (equally in the MVP, with percentage or custom amounts as future extensions) so that costs are divided fairly.

US5. As a group member, I want to see my current balance within each group so that I know immediately who I owe or who owes me.

US6. As a user, I want to receive optimised settlement suggestions so that I can settle all group debts with the minimum number of transactions.

US7. As a group member, I want to receive in-app and on-device notifications when new expenses are added so that I stay informed about group spending

Should Have:

US8. As a user, I want to view spending analytics by category so that I can understand where group money is being spent.

US9. As a group admin, I want to edit or delete expenses so that I can correct mistakes without recreating entries.

US10. As a user, I want to mark settlements as paid so that balances are updated and I have a record of completed payments.

Could Have:

US11. As an international traveller, I want to add expenses in different currencies so that I can track spending during trips abroad.

US12. As a data-conscious user, I want to export expense data so that I can analyse it externally.

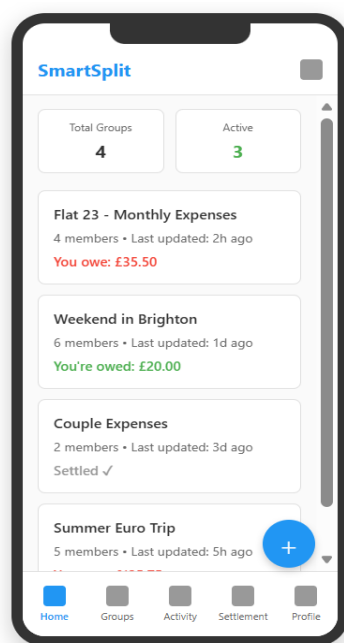
Could-Have stories US11 and US12 were intentionally deferred from the MVP scope and are revisited as future work in Chapter 9.

Three end-to-end user journeys (Angel creating a group and adding the first expense; Tripta viewing balances and settling debts; Dikshya reviewing trip analytics) are documented in Appendix C. Each journey traces a workflow that the named persona would realistically perform and exercises a specific subset of the user stories from Section 4.2.

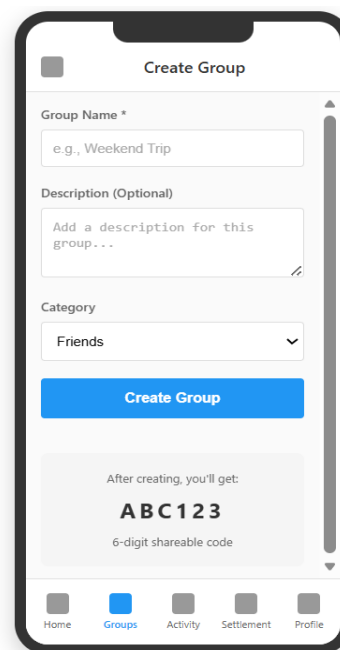
4.3 Wireframes

Five low-fidelity wireframes were produced in Figma to visualise the core user interface before any production code was written: a home screen displaying active groups as

cards with member counts and net balances (Figure 4.1); a two-step group creation flow producing a 6-digit code with sharing options (Figure 4.2); an expense entry screen offering Scan Receipt or Enter Manually paths, with a confirmation step distinguishing auto-filled fields from manually entered ones (Figure 4.3); a settlement view with colour-coded balances and a one-tap Mark as Paid action (Figure 4.4); and an analytics dashboard with category pie chart, monthly bar chart, and ranked spender list (Figure 4.5). Reviewing these wireframes with prospective users surfaced navigation issues that were resolved at the design stage, before they would have become costly to change in code.



*Figure 4.2: Home Screen
Figma wireframe*



*Figure 4.1: Group Creation
Figma wireframe*

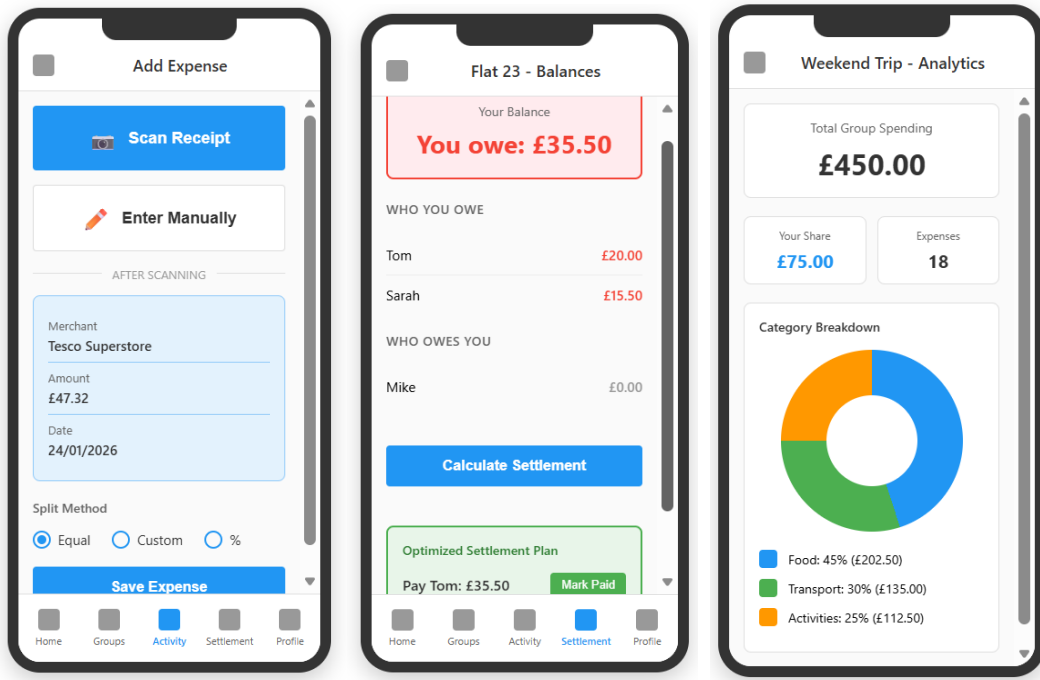


Figure 4.5: Expense Entry Figma wireframe

Figure 4.3: Settlement View Figma wireframe

Figure 4.4: Analytics Dashboard Figma wireframe

Wireframes vs. Implementation: These wireframes capture the original design intent established before development began. The implemented application, shown in screenshots throughout Chapter 6, evolved during construction in response to technical feedback, prototype testing, and minor visual refinements (for example, navigation patterns, control layouts, and colour palette were adjusted after early prototyping). The differences reflect the iterative nature of agile, user-centred development rather than a departure from the original goals, every screen documented above is present in the final application, with the same essential structure and capabilities.

4.4 User Flow Diagram

Figure 4.9 illustrates the primary user flow through SmartSplit's core functionality from registration through expense tracking to settlement with decision points where the interface branches based on user choices.

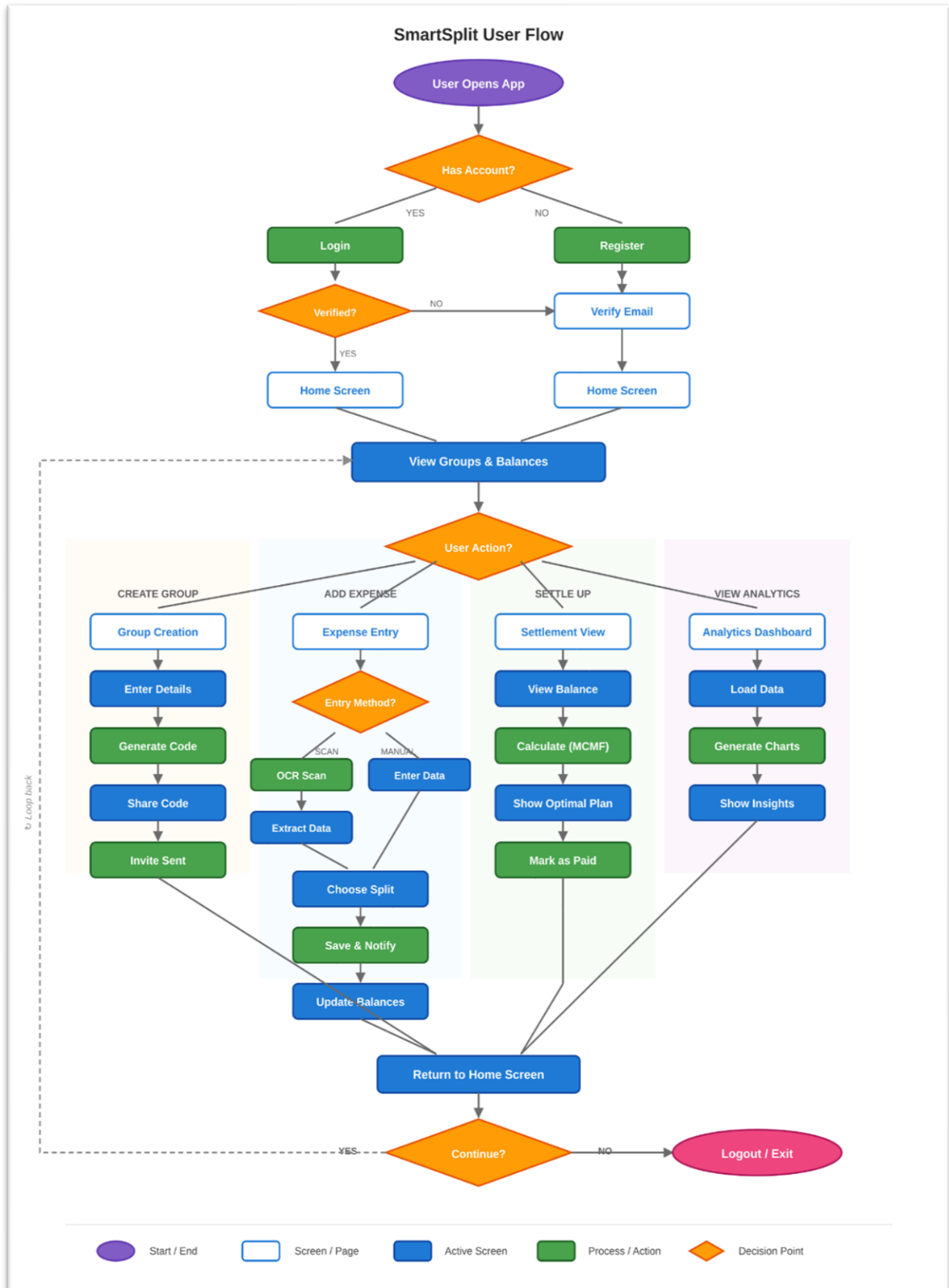


Figure 4.6: Primary user flow through SmartSplit's core workflows

The flow was designed to minimise interaction steps for the most frequent tasks. Adding an expense via OCR scanning, for example, follows the path: **Expense Entry → Entry Method (decision) → OCR Scan → Extract Data → Choose Split → Save & Notify → Update Balances**, replacing what would otherwise be several fields of manual data entry.

4.5 Summary

This chapter has translated the problem identified in Chapters 1 and 2 into concrete requirements through five complementary artefacts: personas representing the target user base, twelve MoSCoW-prioritised user stories capturing functional requirements, three workflow journeys illustrating end-to-end use, Figma wireframes establishing the visual design intent, and a user flow diagram mapping screen transitions and decision points. Chapter 5 presents the technical design that operationalises these requirements.

5 Solution Design

This chapter documents the technical design of SmartSplit through its system architecture, domain model, use cases, database schema, key workflow sequences, and algorithm design. The design operationalises the requirements specified in Chapter 4 and follows established software-engineering principles for modularity, scalability, and maintainability.

5.1 System Architecture

SmartSplit follows a three-tier architecture (Figure 5.1): a Flutter mobile client, a layered service tier of providers and services on the device, and a managed cloud backend on Firebase. The client communicates with Firebase through its SDKs for authentication [14], real-time database operations [12], file storage, and push messaging. Server-side logic that cannot be safely delegated to the client (OTP-based email verification, password reset, and notification dispatch) is implemented as Cloud Functions in Node.js using the Firebase Admin SDK.

Two design choices distinguish this architecture from a conventional thin-client mobile application. First, Google ML Kit operates entirely on-device [10]: receipt images are processed locally and never leave the user's phone, supporting offline use and protecting user privacy. The trade-off, that recognition accuracy on non-standard receipts is weaker than a cloud OCR service could provide [11], is acknowledged in Chapter 7. Second, the MCMF settlement algorithm executes client-side, minimising latency and

avoiding server round-trips for a deterministic computation over data already present on the device.

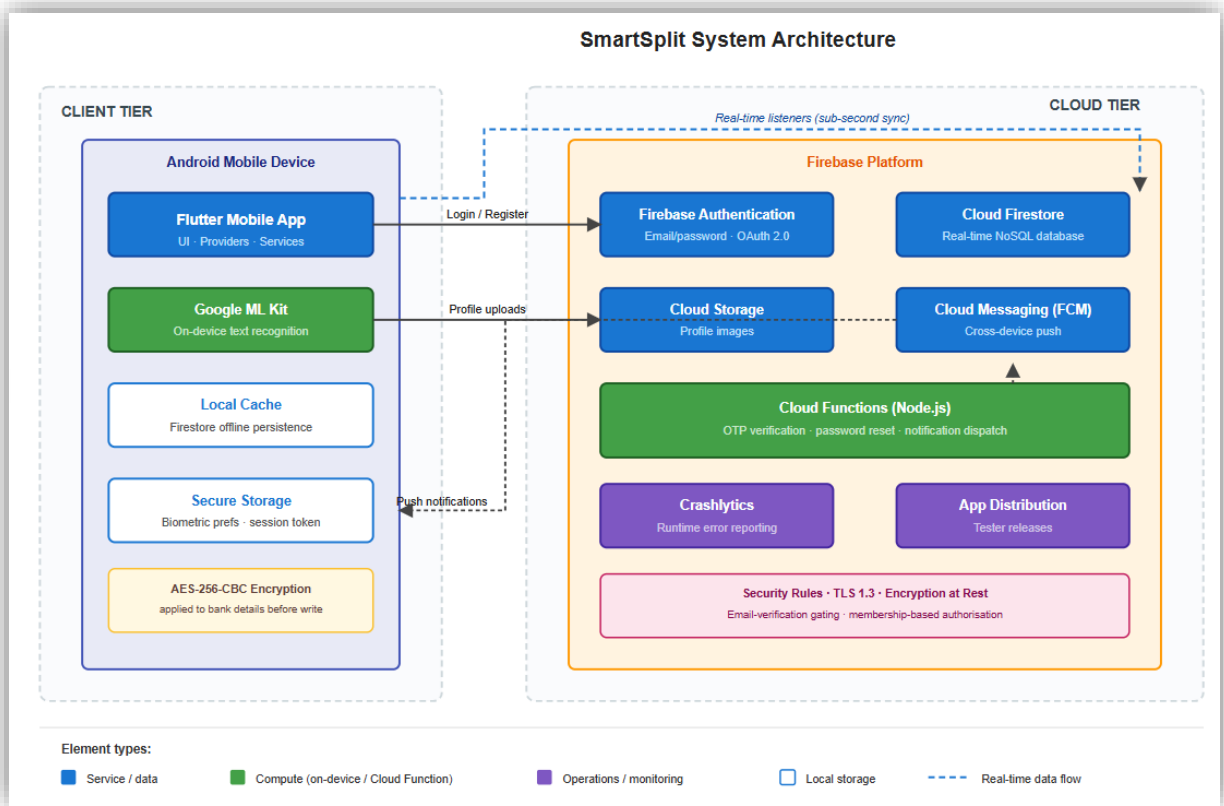


Figure 5.1: Three-tier system architecture: Flutter UI, services and providers, and the Firebase backend with on-device ML Kit

5.2 Domain Model

Seven model classes encapsulate the persisted domain (Figure 5.2): `UserModelID`, `GroupModel`, `ExpenseModel` with embedded `ExpenseSplit` objects, `SettlementRecord`, `BankDetail`, `NotificationModel`, and `GroupAnalytics`. Receipt scanning is represented by a transient data-transfer object (`ScannedReceiptDat`) rather than a persisted entity; balances are computed on demand rather than stored, eliminating the synchronisation burden of maintaining derived data.

Relationships include a one-to-many between groups and expenses, a one-to-many between groups and settlements, and a many-to-many between users and groups expressed via a **member** array on each group document. Per-user notifications and per-user bank details are nested under each user record. The choice to embed splits within an expense, rather than store them as a separate entity, exploits Firestore's document-model strengths [13] and reduces the number of read operations required for balance calculation.

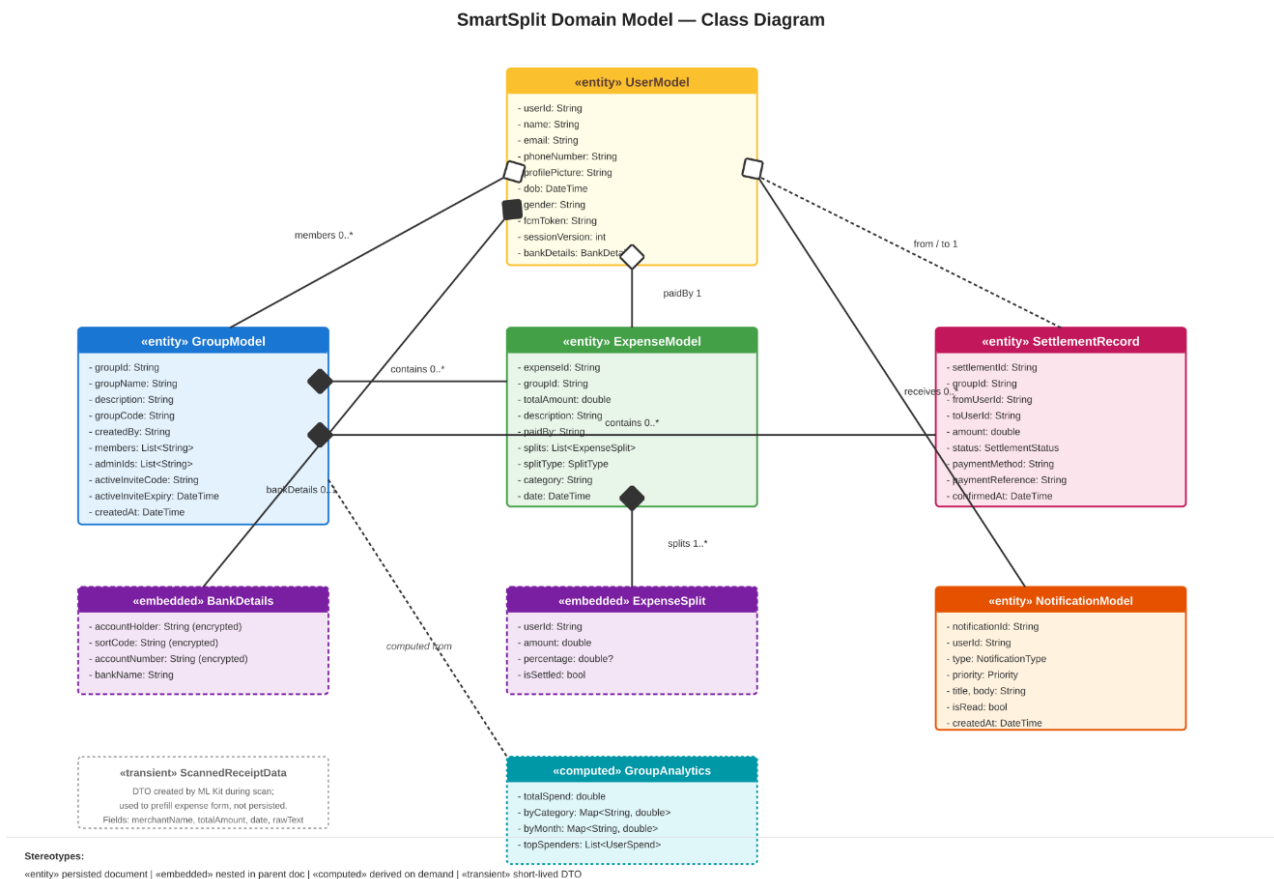


Figure 5.2: Class diagram showing the seven domain models and their relationships

5.3 Use Cases

Figure 5.3 presents the use case diagram. Four actors are identified : User (primary), Firebase Authentication, Cloud Firestore, and Google ML Kit and the system

functionality is partitioned into five packages: Authentication and Profile, Group Management, Expense Management, Settlement, and Analytics and Notifications. The use cases map directly to the user stories defined in Chapter 4: each Must-Have story corresponds to one or more use cases in the diagram, providing visible traceability from the requirements specification to the design.

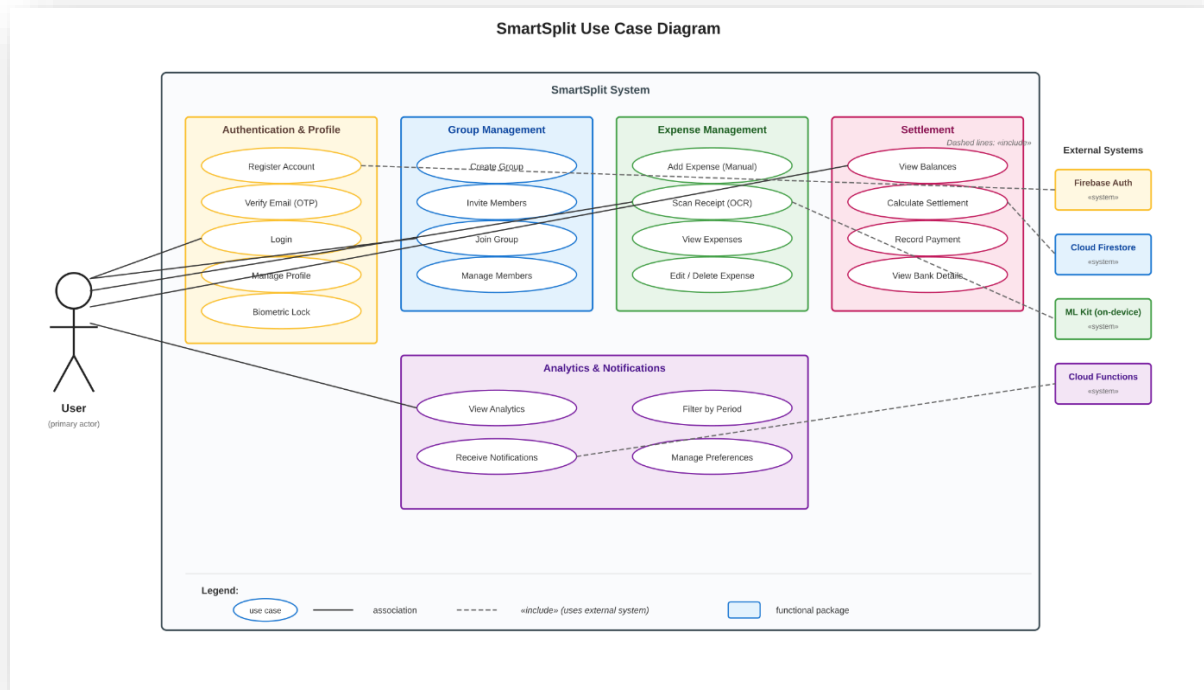


Figure 5.3: Use case diagram showing four actors and five functional packages

5.4 Database Design

SmartSplit uses Cloud Firestore [12], a document-oriented NoSQL database. The schema (Figure 5.4) differs from a relational design in three important ways: subcollections nest under their parent documents, embedded objects (such as ExpenseSplit) reduce the need for join operations, and arrays (such as members) replace simple junction tables. This document-oriented design is well-suited to the nested-group, nested-expense structure of the application and allows real-time listeners to track changes at any level of the tree.

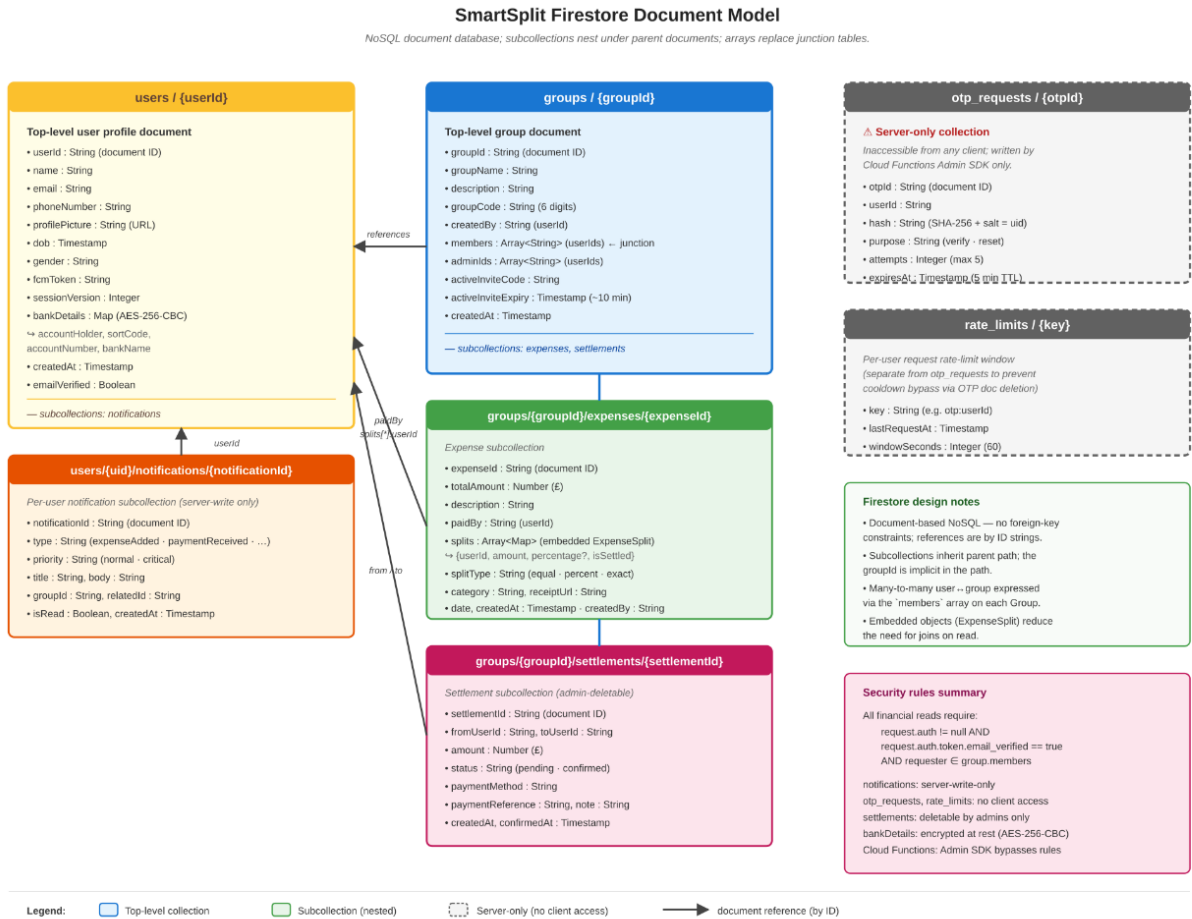


Figure 5.4: Firestore document model showing collections, subcollections, and embedded structures

Table 5.1: Firestore collections

Path	Purpose	Notable fields
<code>users/{userId}</code>	User profile	<code>name</code> , <code>email</code> , <code>phoneNumber</code> , <code>fcmToken</code> , <code>bankDetails</code> (encrypted), <code>sessionVersion</code>
<code>users/{userId}/notifications/{id}</code>	Per-user notifications	<code>type</code> , <code>priority</code> , <code>title</code> , <code>body</code> , <code>isRead</code> , <code>createdAt</code>
<code>groups/{groupId}</code>	Group document	<code>groupName</code> , <code>groupCode</code> , <code>members[]</code> , <code>adminIds[]</code> , <code>activeInviteCode</code>
<code>groups/{groupId}/expenses/{id}</code>	Expense subcollection	<code>totalAmount</code> , <code>paidBy</code> , <code>splits[]</code> , <code>splitType</code> , <code>category</code> , <code>date</code>

<code>groups/{groupId}/settlements/{id}</code>	Settlement subcollection	<code>fromUserId</code> , <code>toUserId</code> , <code>amount</code> , <code>status</code> , <code>paymentMethod</code> , <code>note</code>
<code>otp_requests/{otpId}</code>	Server-only OTP verification	Salted SHA-256 hash, attempts counter, expiry timestamp

The schema includes two design choices worth surfacing. The `paidBy` field allows any member to record an expense paid by another on their behalf, and a `per-split included` flag lets a single expense be apportioned among a chosen subset of group members. Together these make the schema more expressive than equal-split alone suggests.

Firestore security rules [15] enforce defence-in-depth authorisation. Reads of group, expense, and settlement documents require both authentication and email verification, with group membership verified through a `get()` reference. Settlements may be created and read by any verified member but deleted only by group administrators, and the `otp_requests` collection is accessible only to the Cloud Functions Admin SDK. Combined with encryption at rest, financial data is protected by multiple independent mechanisms, an approach grounded in established authentication design [17].

5.5 Key Workflow Sequences

Two sequence diagrams illustrate the most algorithmically distinctive workflows: OCR-driven expense addition (Figure 5.5) and settlement calculation (Figure 5.6). The detailed implementation of each workflow is documented in Sections 5.6.1 and 5.6.2 respectively. User registration, group creation, and real-time synchronisation follow conventional Firebase patterns and are not depicted separately.

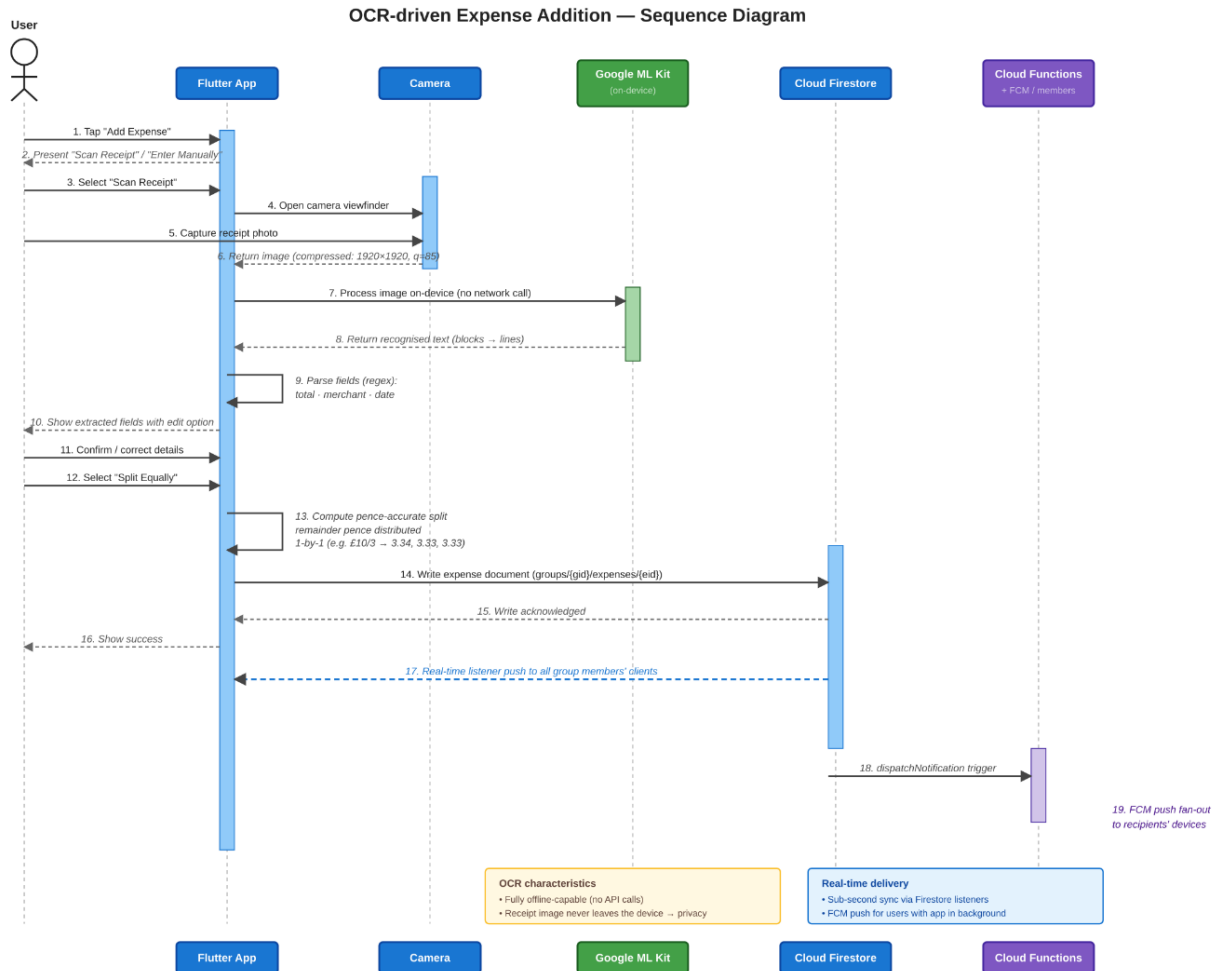


Figure 5.5: OCR Expense Addition — sequence diagram

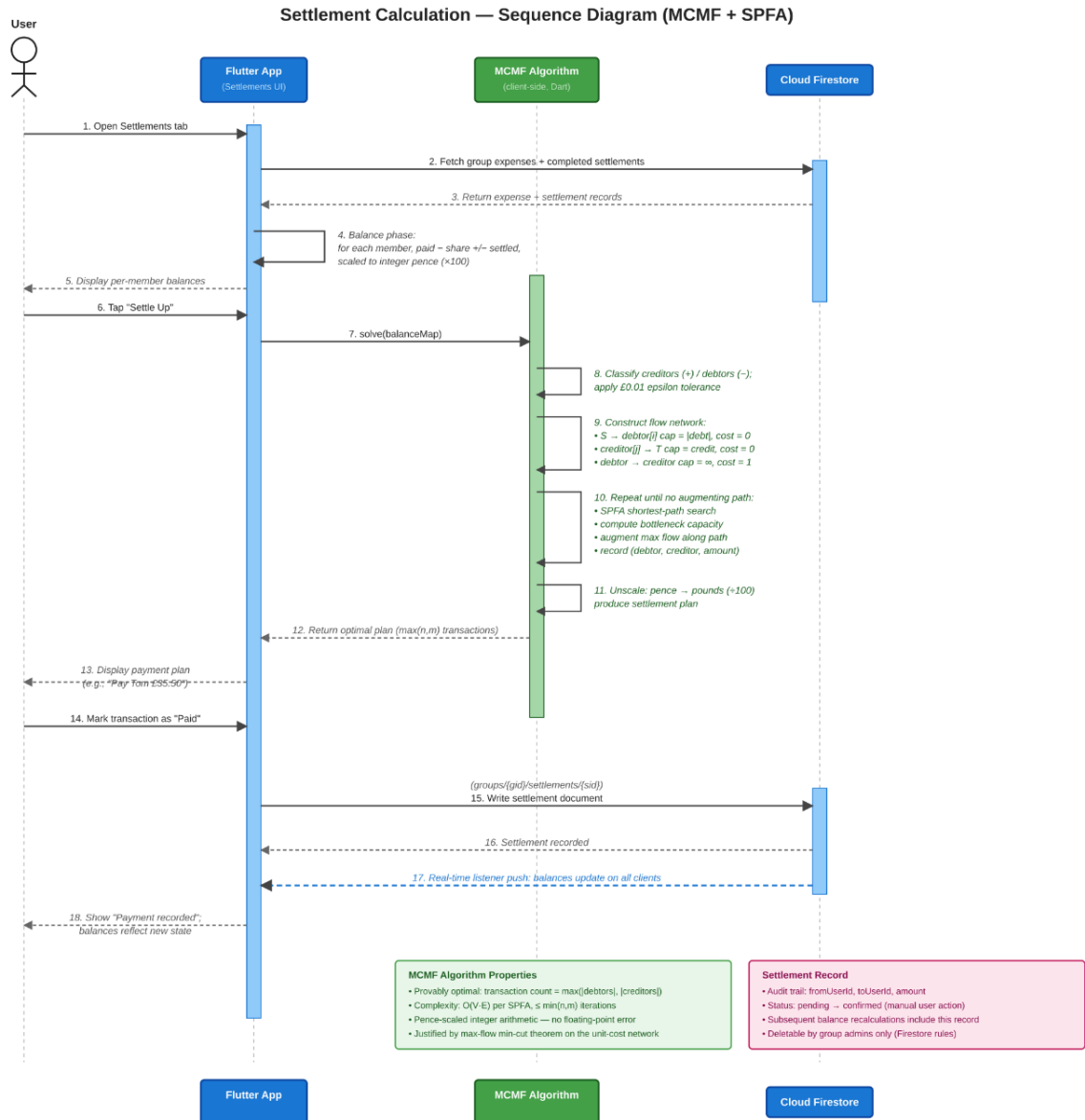


Figure 5.6: Settlement Calculation — sequence diagram

5.6 Algorithm Design

5.6.1 OCR Text Extraction

ML Kit's text-recognition API returns recognised text as a hierarchical structure of blocks, lines, and elements. The parser flattens this into a list of lines and applies pattern matching to extract three target fields: total amount, transaction date, and

merchant name. Totals are detected by bottom-up keyword scanning (e.g. "total", "amount due"), with the largest currency-formatted value as a fallback, an approach informed by structured-extraction strategies in the literature [8]. Dates match standard UK and ISO formats with calendar-plausibility checks; merchants are taken from the first non-address, non-date lines.

A regex-and-heuristics approach was chosen over a learned model for code simplicity and predictable behaviour: a small, inspectable parser is easier to debug and evolve than a black-box classifier when failures are reported during testing. The cost, that the parser cannot generalise to receipt layouts it has not been hand-engineered for, is reported in Chapter 7.

5.6.2 Settlement Optimisation

Each member's net balance is computed as the difference between what they paid and what they owe across expenses they participated in, with completed settlements applied as adjustments. Members with positive balance are creditors; those with negative balance are debtors.

The optimisation problem is modelled as a Minimum-Cost Maximum-Flow (MCMF) network [21]: a source S connects to each debtor with capacity equal to the debt and zero cost; each creditor connects to a sink T with capacity equal to the credit and zero cost; and every debtor-creditor pair is connected with unbounded capacity at unit cost. Because the only non-zero-cost edges are the debtor-to-creditor edges, minimising total flow cost is equivalent to minimising the number of distinct transactions. The algorithm applies successive shortest-path augmentation through SPFA (the Bellman-Ford queue variant) with max-bottleneck augmentation [18], recording each augmenting path as one settlement. Overall complexity is $O(n^2m^2 + nm)$, readily practical for typical consumer expense-sharing group sizes.

The choice of MCMF over a greedy heuristic is deliberate. The general debt-settlement problem is NP-complete [19], and greedy approaches such as iterative largest-creditor /

largest-debtor matching [20] cannot guarantee optimality. The MCMF formulation produces a transaction count equal to $\max(n, m)$, the theoretical minimum for any balanced debt graph, at tractable cost. The guarantee is grounded in the max-flow min-cut theorem [21].

Figure 5.7 illustrates the algorithm on a four-member group with two creditors and two debtors. The naive baseline requires four transactions; MCMF reduces this to two, matching the theoretical minimum. Gains are largest when debtors and creditors pair cleanly; in less symmetric cases the reduction relative to the naive baseline is smaller, though MCMF still finds the minimum.

Balances are processed as integer pence throughout, eliminating floating-point error; an epsilon tolerance of £0.01 prevents tiny rounding artefacts from generating spurious zero-pence transactions.

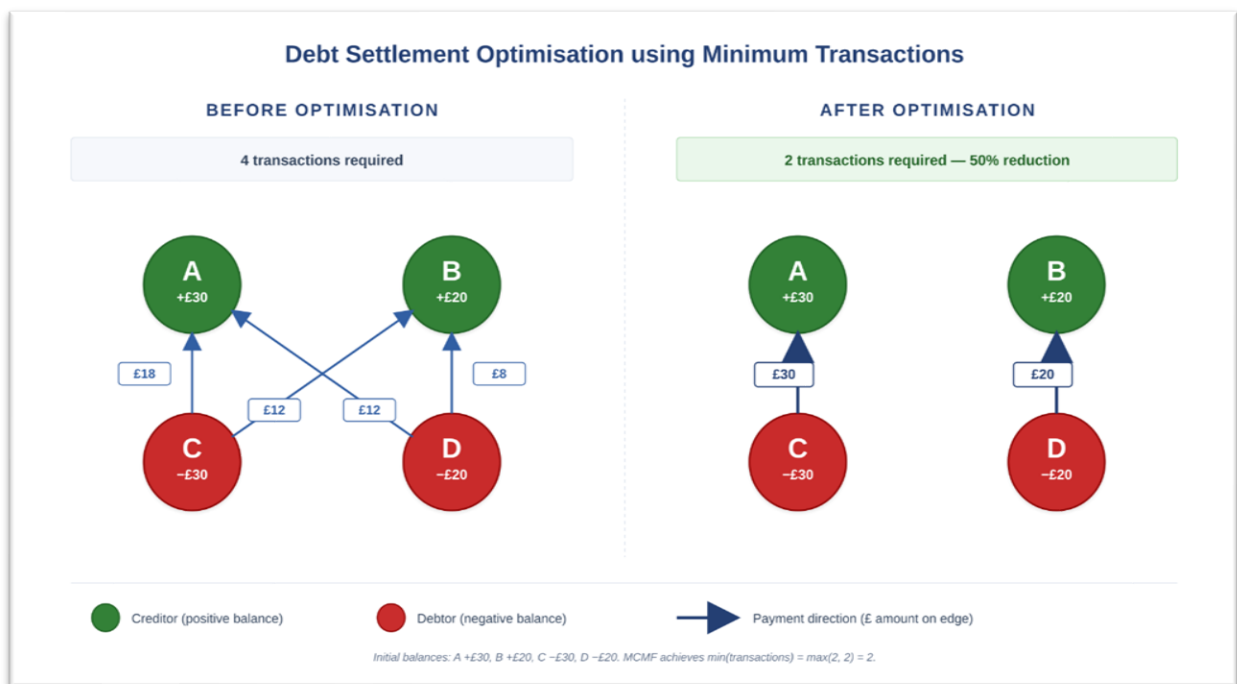


Figure 5.7: Debt settlement optimisation using minimum transactions

Balances are scaled by 100 and processed as integer pence throughout, eliminating floating-point error from financial sums; an epsilon tolerance of £0.01 prevents tiny rounding artefacts from generating spurious zero-pence transactions.

5.7 Summary

This chapter has documented SmartSplit's technical design: a three-tier system architecture, a seven-class domain model expressed in Cloud Firestore's document schema, a use case diagram traceable to the requirements of Chapter 4, the database schema with its security rules, sequence diagrams for the OCR and settlement workflows, and the algorithm design for OCR text extraction and MCMF-based settlement. The MCMF settlement algorithm is provably optimal under the max-flow min-cut theorem [21], a stronger guarantee than the greedy heuristics found in competing applications. Chapter 6 documents the implementation of this design.

6 Solution Implementation

This chapter documents SmartSplit's implementation across six development cycles. Each section describes the deliverables of one cycle and the engineering decisions worth surfacing. Code excerpts are limited to two distinctive components, the OTP hashing flow and the MCMF flow-network construction, with the full source available at github.com/boharap1/SmartSplit.

6.1 Development Environment

Development used Flutter 3.38.4 with Dart 3.10.3 on Windows 11, with Visual Studio Code as the editor and Android Studio providing the SDK and emulator. The Firebase project was configured with Authentication [14], Cloud Firestore [12], Cloud Functions, Cloud Messaging, Crashlytics, and App Distribution enabled. An early Gradle compatibility issue between Flutter's default Android Gradle Plugin and the Firebase BoM dependencies was resolved by updating to AGP 8.9.1, the kind of transitive-dependency conflict anticipated in the project risk register (R4).

6.2 Sprint 1: Authentication and Account Security

Sprint 1 delivered email/password registration and sign-in through Firebase Authentication [14], plus three OTP flows (pre-registration email verification, password reset, and account-deletion confirmation) implemented as Cloud Functions in Node.js.

OTPs are stored as salted SHA-256 hashes (salt = user UID) so that a leak of the `otp_request` collection cannot be inverted; verification uses constant-time comparison through `crypto.timingSafeEqual` to defend against timing-side-channel attacks (Listing 6.1). Rate limiting runs on a separate `rate_limits` collection through atomic Firestore transactions, preventing an attacker from bypassing the 60-second cooldown by

deleting their OTP document between requests. The password-reset endpoint returns success regardless of whether the email is registered, defeating account enumeration. Together these implement the layered authentication model recommended by Bonneau et al. [17].

The OTP pipeline also performs a DNS MX-record lookup before SMTP submission, rejecting unroutable addresses synchronously, and rolls back both the OTP and rate-limit documents atomically if SMTP delivery fails. Multi-device session revocation is supported through a server-tracked `sessionVersion` field that clients compare against on each app resume, forcing a local sign-out on mismatch. A `getIdToken(true)` force-refresh handles the Firebase quirk where cached ID tokens continue to claim `email_verified: false` for up to an hour after server-side verification.

Listing 6.1: OTP hash generation and constant time verification

(`functions/index.js`)

```
const hash = crypto.createHash('sha256')
  .update(`smartsplit:${uid}:${otp}`)
  .digest('hex');

// Constant-time comparison – defends against timing
// attacks
const match = crypto.timingSafeEqual(
  Buffer.from(stored.hash, 'hex'),
  Buffer.from(submitted, 'hex'),
);
```

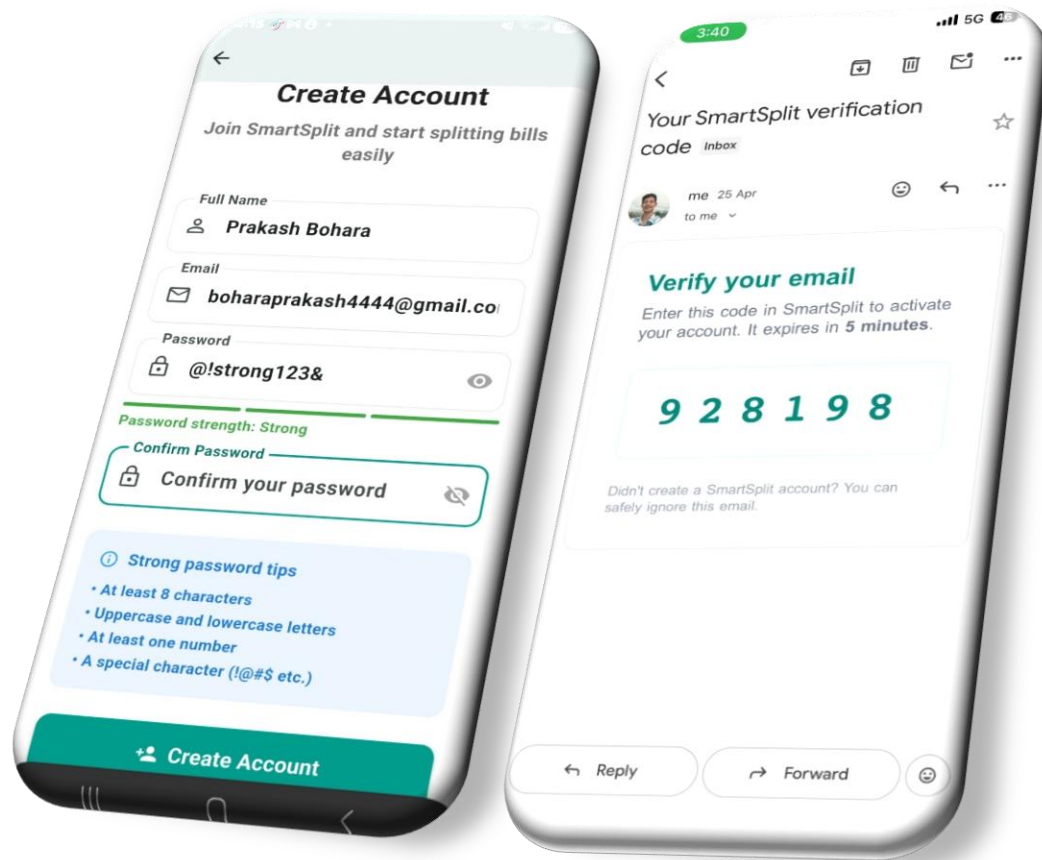


Figure 6.1: Authentication screens — registration with strength indicator, OTP entry, and email verification

6.3 Sprint 2: Group Management

Sprint 2 delivered group creation, member invitation, and real-time group listing. Each group has a name, optional description, icon, colour, and currency, and is identified by an auto-generated 6-digit join code produced via a randomisation routine with collision checking against existing codes. Members can be invited either by email or by sharing the join code directly.

The `GroupProvider` subscribes to a Firestore snapshot stream filtered to groups containing the current user's ID, ensuring the home-screen group list updates in real time across devices when members join, leave, or rename groups. Owner-only operations (delete group) are enforced both in the UI and in `firestore.rules` via membership checks

against the parent document, a server-side check that cannot be circumvented by a tampered client.

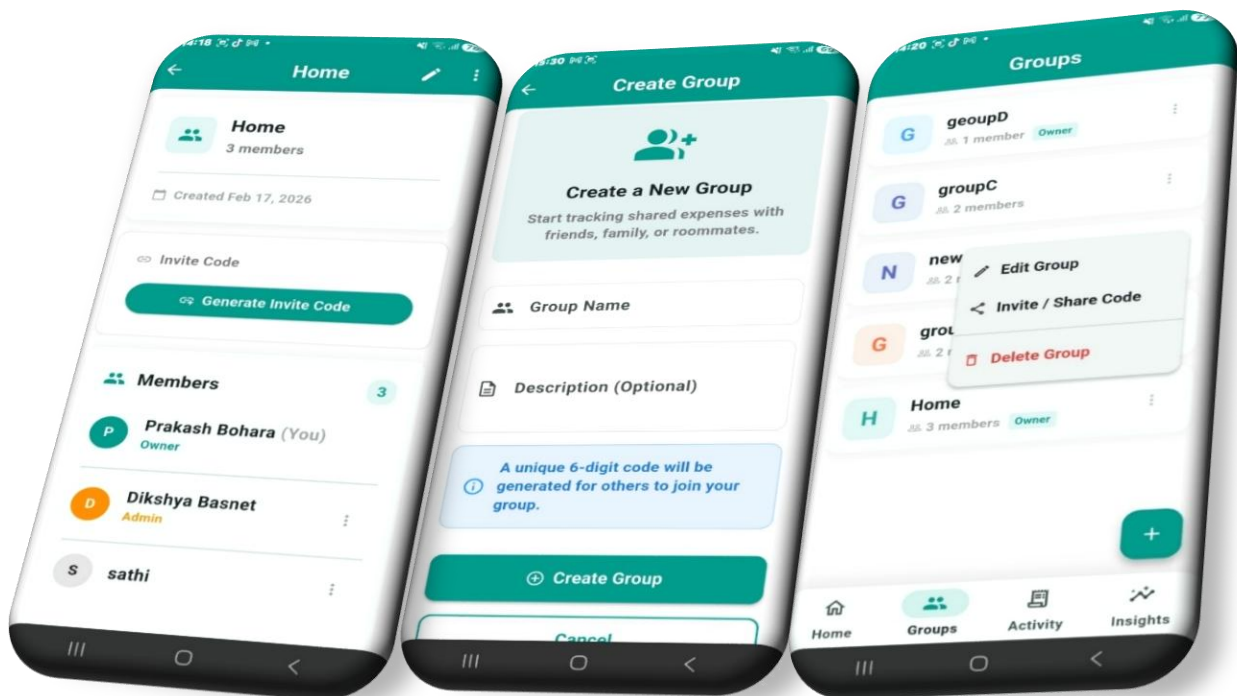


Figure 6.2: Group creation screen and group details screen showing member list and balances

6.4 Sprint 3: Expense Management with OCR

Sprint 3 delivered manual expense entry, on-device OCR receipt scanning, and the balance-calculation engine.

Expenses are recorded with a description, amount, date, payer, participants, and one of six categories (Food, Transport, Accommodation, Entertainment, Shopping, Utilities). Each expense supports per-member inclusion (a checkbox excludes individual members from this bill) and pay-on-behalf (any member can record an expense paid by another). Equal-split distribution uses penny-accurate integer arithmetic to avoid floating-point error: amounts are scaled to pence, divided among participants, and any

remainder is distributed one pence at a time. For example, £10.00 split three ways produces shares of £3.34, £3.33, and £3.33 — summing exactly to the original total.

OCR receipt scanning is implemented through the `google_mlkit_text_recognition` package [10], which performs all recognition on-device. The user captures a receipt photo through the in-app camera; ML Kit returns recognised text blocks; and a parsing routine (Section 5.6.1) extracts merchant, total, and date for user confirmation before the expense is committed. No image leaves the phone, supporting both privacy and offline operation. The recognition accuracy on standard supermarket receipts was reliable in functional testing (Section 7.3); accuracy on faded thermal-printed and unusually formatted receipts is weaker, as anticipated by the OCR literature on receipt processing [9].

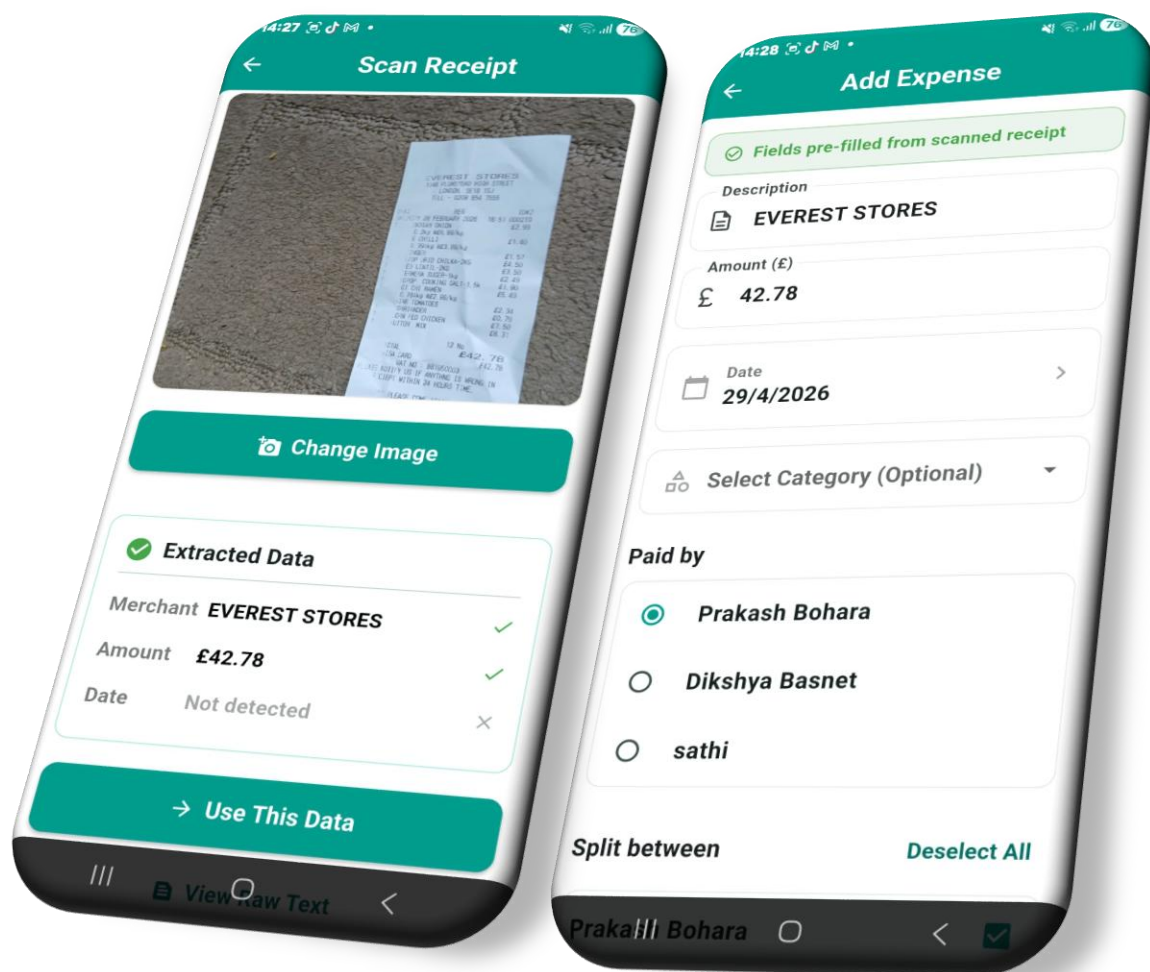


Figure 6.3: Scan Receipt screen with extracted data preview and confirmation

The expense list is paginated using Firestore's `.limit` queries with a "Load more" button (20 items per page), keeping the UI responsive on groups with hundreds of historical expenses. Balance recalculation runs on every change, with complexity $O(E \cdot S)$ where E is the number of expenses and S the average number of participants per expense.

6.5 Sprint 4: Settlement Optimisation and Bank Details

Sprint 4 delivered the settlement-optimisation algorithm and encrypted bank-detail storage.

Settlement. The algorithm formulates the settlement problem as a flow network (Listing 6.2) and applies SPFA-based MCMF to find the minimum-transaction settlement, as documented in Section 5.6.2. The optimised plan is presented per group on the Settlements screen, showing each recommended payment with bank-detail expansion for direct transfer. Recording a payment writes a settlement document to `groups/{id}/settlements`, which is reflected in updated balances via the Firestore snapshot listener.

Listing 6.2: MCMF flow-network construction

(`settlement_algorithm.dart` , abridged)

```
// Source → debtor edges (capacity = debt in pence, cost = 0)
for (var i = 0; i < debtors.length; i++) {
  graph.addEdge(SOURCE, debtorNodes[i], cap: debts[i], cost: 0);
}
// Creditor → sink edges (capacity = credit in pence, cost = 0)
for (var j = 0; j < creditors.length; j++) {
  graph.addEdge(creditorNodes[j], SINK, cap: credits[j], cost: 0);
}
// Debtor → creditor edges (capacity = ∞, cost = 1)
for (var i = 0; i < debtors.length; i++) {
  for (var j = 0; j < creditors.length; j++) {
    graph.addEdge(debtorNodes[i], creditorNodes[j], cap: INF, cost: 1);
  }
}
```

Bank-detail encryption. Account number, sort code, and account holder name are encrypted at rest using AES-256-CBC: the encryption key is derived as SHA-256(server_pepper + user_uid), with a fresh random IV generated on every save. This means the same plaintext produces different ciphertexts on each write, defeating frequency analysis, while still allowing legitimate decryption when the user opens their own bank-details screen.

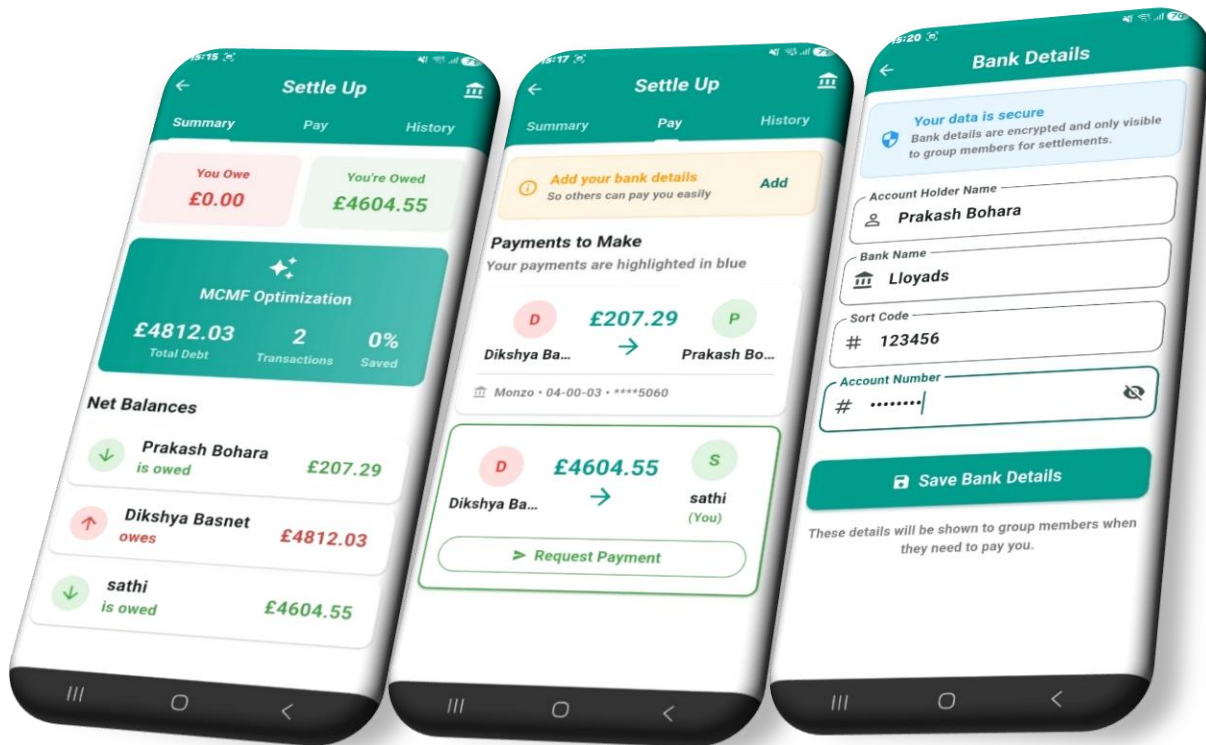


Figure 6.4: Settlement plan screen showing recommended payments and bank-detail expansion

A late-discovered `PERMISSION_DENIED` error during integration was traced to missing Firestore rules for the `settlements` subcollection, the rules originally covered the parent group but not its children. This was resolved by extending `firestore.rule` with explicit `match/settlements/{settlementId}` blocks that mirror the parent's membership and email-verification checks.

6.6 Sprint 5: Notifications

Sprint 5 delivered an in-app notification centre and cross-device push delivery via Fire-base Cloud Messaging (FCM).

In-app notifications are stored in `users/{uid}/notifications` and rendered through a Firestore snapshot listener, with a badge count on the navigation tab. Cross-device delivery uses a `dispatchNotification` Cloud Function that validates group membership server-side, writes notification documents via the Admin SDK (bypassing client rules), and sends an FCM push to each recipient's stored device token. The Firestore rule `allow create: if` on the notifications subcollection ensures that only the server-side path can create notifications, clients cannot inject notifications into other users' in-boxes.

When the application is in the foreground, FCM messages are intercepted and rendered as local heads-up notifications via `flutter_local_notifications`. A notification preferences screen exposes per-type toggles (expense added, payment received, payment request, group invitation, group leave, settlement marked as paid).

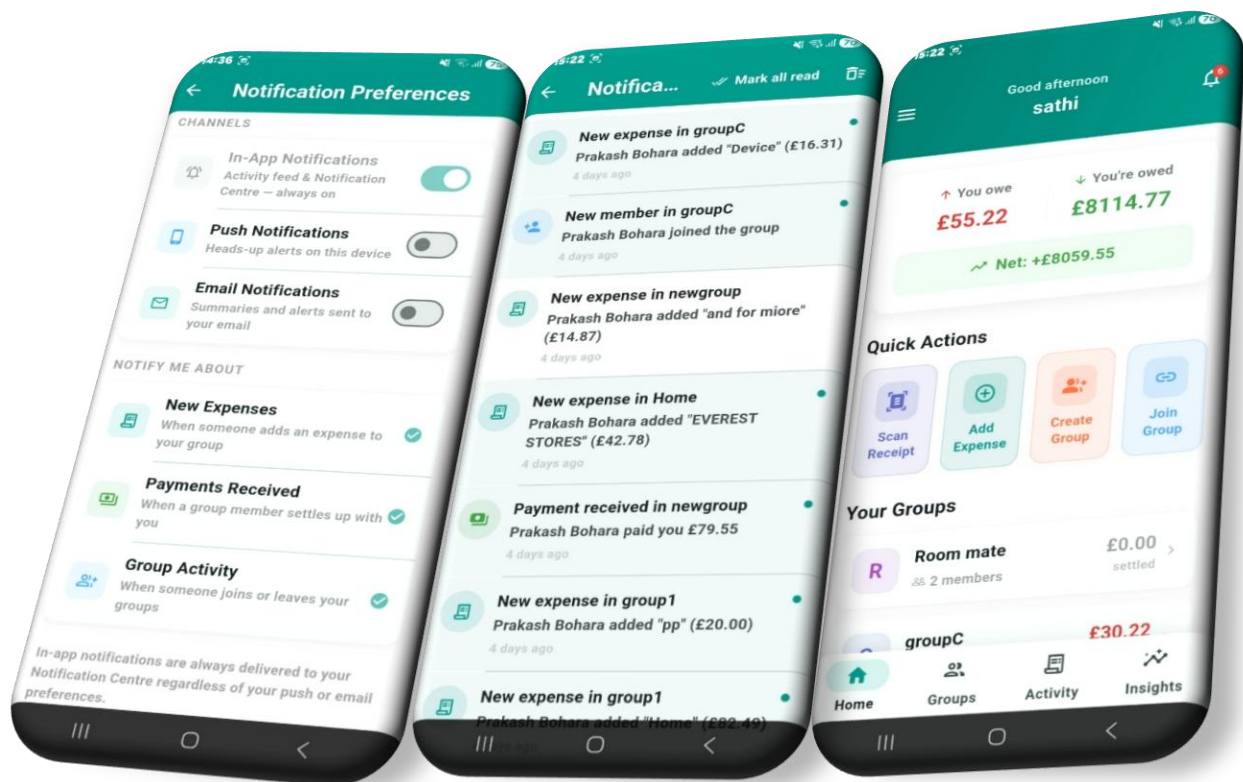


Figure 6.5: Notification centre with badge counter and notification preferences screen

6.7 Sprint 6: Analytics, Settings, and Profile

Sprint 6 delivered the analytics dashboard, application settings, and the profile management surface.

The group analytics screen presents total spend, expense count, average expense, a category breakdown (pie and bar charts via `fl_chart`), a monthly spending trend, and per-member contribution breakdown. An Activity tab complements this with a unified cross-group timeline of expenses and settlements, grouped by date and refreshed through pull-to-refresh.

Settings include a currency selector with the chosen symbol propagated app-wide through shared preferences, light/dark/system theme, timezone display, and language preference. Profile management covers name and avatar editing, personal information, bank-details management (which writes through the AES-256-CBC encryption layer described in Section 6.6), and account-verification status.

A biometric app-lock is implemented through `local_auth`: on app resume, if the user has enabled the lock, the device prompts for fingerprint, face, or device PIN before the UI is revealed. The preference is persisted through `flutter_secure_storage` so it cannot be cleared by malicious access to standard preferences. The Password and Security screen requires current-password re-authentication before changing the password, protecting against an attacker with brief access to an unlocked device.



Figure 6.7: Analytics dashboard with category breakdown chart, monthly trend and top contributor

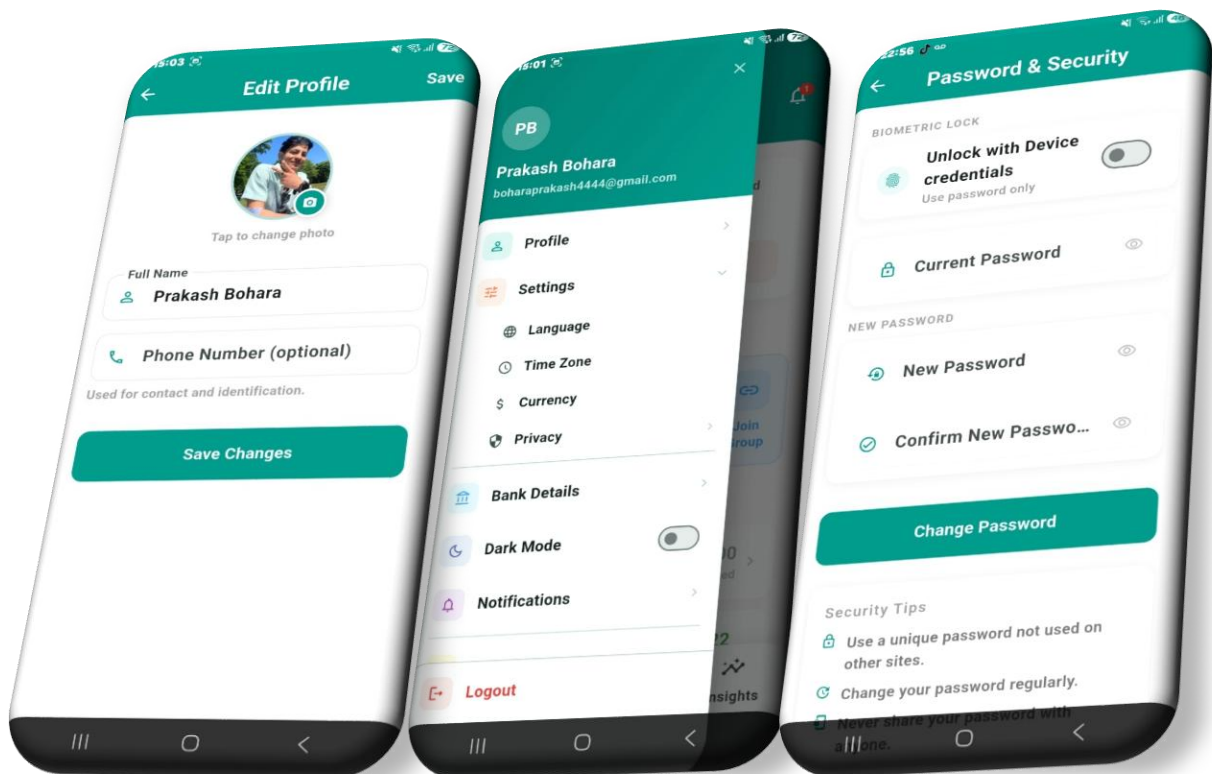


Figure 6.6: profile screen, Settings screen, password biometric and security screen

6.8 Cross-cutting Concerns

Three cross-cutting capabilities support the application as a whole. **Offline persistence** is enabled through Firestore with `persistenceEnabled:` allowing the application to read from a local cache when offline and reconcile writes once connectivity returns. Error reporting uses Firebase Crashlytics wired into `FlutterError.onError` and `PlatformDispatcher.instance.onError;` service-layer catch blocks log through a centralised helper that forwards non-fatal exceptions to Crashlytics with relevant context. Firestore security rules [15] apply tiered authorisation: OTP documents are accessible only to the Admin SDK; user profiles permit owner-only writes; group, expense, and settlement reads require both authentication and email verification; and notifications are server-write-only.

The release pipeline uses Firebase App Distribution with the Gradle plugin: each `appDistributionUploadRelease` run uploads a signed APK and notifies registered testers automatically, supporting the iterative user-acceptance testing described in Chapter 7.

6.9 Implementation Summary

Sprints 1–6 are complete. Table 6.1 summarises the implementation across all sprints. The application implements MCMF settlement optimisation with provable transaction-count optimality, on-device OCR receipt scanning, server-side OTP authentication with hashed storage and atomic-transaction rate limiting, AES-256-CBC encrypted bank details, real-time multi-device synchronisation with offline persistence, and cross-device FCM push notifications dispatched through Cloud Functions. Known limitations: a UI restricted to equal splits, OCR accuracy dependent on receipt clarity, and unit-test coverage focused on the settlement algorithm are evaluated in Chapter 7.

Table 6.1: Sprint summary

Sprint	Focus area	Key components implemented	Status
--------	------------	----------------------------	--------

1	Authentication and account security	Firebase Auth; OTP flow via Cloud Functions; salted SHA-256 hashing; atomic-transaction rate limiting; biometric app-lock	Complete
2	Group management	Group CRUD; 6-digit join codes; member invitation; real-time group listing	Complete
3	Expense management with OCR	Manual expense entry; on-device ML Kit OCR; penny-accurate equal-split engine; paginated expense list	Complete
4	Settlement optimisation and bank details	MCMF + SPFA settlement algorithm; AES-256-CBC encrypted bank details; settlement history	Complete
5	Notifications	In-app notification centre; FCM cross-device push via <code>dispatchNotification</code> Cloud Function; preference toggles	Complete
6	Analytics, settings, and profile	<code>f1_chart</code> analytics dashboard; cross-group activity feed; theme/currency/timezone settings; profile + bank-details management	Complete

7 Solution testing and Evaluation

This chapter evaluates SmartSplit through four complementary methods: unit testing of the settlement algorithm, manual functional testing per sprint, security testing of authentication and data-at-rest mechanisms, and a pilot user-acceptance study. The strengths and weaknesses revealed are summarised in Section 7.6.

7.1 Testing Strategy

A multi-layered testing strategy was adopted, reflecting both the financial sensitivity of the application and its user-facing nature. Four layers were applied iteratively across the Agile sprints:

1. **Unit testing** of the settlement algorithm: the most algorithmically complex and financially consequential component.
2. **Functional testing** of each feature per sprint, on real Android devices, against the user stories from Chapter 4.
3. **Security testing** focused on authentication, authorisation, and data-at-rest protection.
4. **User acceptance testing** with representative users, distributed via Firebase App Distribution with structured feedback collected through a Google Form.

Each layer is described in turn, and Section 7.6 summarises the strengths and weaknesses uncovered.

7.2 Unit Testing

The settlement algorithm was the only component to receive automated unit-test coverage. This decision was deliberate: it is the most algorithmically complex piece of logic in the application, the one directly responsible for financial accuracy, and the one whose failure modes are hardest to detect by manual inspection alone (a one-pence

rounding bug, for example, may not be visible to a user but compounds across many settlements).

The test suite (test/settlement_algorithm_test.dart) comprises nine cases written against the Flutter testing framework. Table 7.1 summarises them.

Table 7.1: Settlement algorithm test suite

#	Test case	Purpose	Result
1	Two-user simple debt	One debtor, one creditor → exactly one transaction	Pass ✓
2	Three-user optimisation	Verifies minimum transaction count	Pass ✓
3	All-zero balances	No payments needed	Pass ✓
4	Near-zero (epsilon tolerance)	£0.005 imbalance must not produce a transaction	Pass ✓
5	Penny-rounding correctness	All output amounts have ≤ 2 decimal places	Pass ✓
6	Multi-creditor / multi-debtor	All debts settled exactly	Pass ✓
7	Single-user group	Zero transactions	Pass ✓
8	Transaction-count reduction	Optimised count $\leq$ naive count	Pass ✓
9	Unbalanced totals detection	Detect $\sum \text{debits} \neq \sum \text{credits}$ and report error	Pass ✓

```

PS D:\Projects\smartsplit> flutter test test/settlement_algorithm_test.dart
00:08 +9: All tests passed!
PS D:\Projects\smartsplit>

```

Figure 7.1: Test runner output showing all 9 tests passing

This level of unit-test coverage was intentionally limited in scope. More comprehensive unit testing of the UI and service layers was not undertaken, which is acknowledged as a limitation in Section 7.6 and discussed further as an area for future work in Chapter 9.

7.3 Functional Testing

Functional testing was conducted manually across each sprint on a Samsung Galaxy A16 5G (Android 14) and the Android Studio emulator (Pixel 6, Android 13) in addition to the Android emulator. Tests followed the user stories from Chapter 4. Table 7.2 maps a representative subset of functional requirements to test outcomes.

Table 7.2: Functional testing results, mapped to requirements

Req.	Feature tested	Outcome
FR1	Email/password registration and login	Pass 
FR2	OTP email verification	Pass 
FR3	OTP password reset	Pass 
FR4	Group creation with auto-generated 6-digit code	Pass 
FR5	Join group by code or email invitation	Pass 
FR6	Add expense (manual entry)	Pass 
FR7	Equal-split with selective inclusion and pay-on-behalf	Pass  , verified with awkward divisions (e.g. $\pounds 10 \div 3 \rightarrow \pounds 3.34, \pounds 3.33, \pounds 3.33$)
FR8	OCR receipt scanning	Pass with caveats  , see section 7.6
FR9	Real-time balance update across devices	Pass 
FR10	Settlement plan generation	Pass 
FR11	Record completed payment	Pass 
FR12	Bank-detail storage (encrypted)	Pass 
FR13	Notification dispatch (in-app + push)	Pass 
FR14	Analytics dashboard	Pass 
FR15	Offline persistence	Pass  , verified by air-plane-mode toggle

OCR (FR8) is recorded as *Pass with caveats*: extraction of the total amount and merchant name was reliable on standard supermarket receipts, but accuracy degraded on faded thermal-printed café receipts and on receipts where the total appeared in unusual layouts. This is consistent with the limitations of regex-based heuristic parsing acknowledged in Section 5.4.1, and is discussed further in Section 7.6.

7.4 Security Testing

Three categories of security mechanism were validated.

OTP rate limiting. The atomic-transaction implementation on the `rate limit` collection (Section 6.2) was reviewed by inducing rapid successive OTP requests during development. The cooldown logic correctly rejected requests inside the 60-second window and could not be bypassed by deleting the OTP document between requests, because the rate-limit record is held in a separate collection.

Firestore security rules. Rules were inspected by manual probing through authenticated and unauthenticated clients. Specific scenarios verified included: an unverified user attempting to read group data (correctly denied); a non-member attempting to read another group's expenses (denied); a member attempting to write a notification document directly (denied — only the server-side Admin SDK path is permitted); and any attempt to read the `otp_request` collection from a client (denied).

Encryption at rest. Bank details written via the Profile screen were inspected directly in the Firestore console. The stored ciphertext was verified to differ between two saves of identical plaintext, confirming that the random IV is being applied correctly per write, defeating frequency-analysis attacks against the encrypted column.

No tested scenario revealed an exploitable weakness in the implemented mechanisms. However, the testing scope was deliberately limited: independent penetration testing, formal threat modelling, and dependency vulnerability scanning are all acknowledged in Section 7.6 as out of scope for this project.

7.5 User Acceptance Testing

User acceptance testing was conducted as a small pilot study using Firebase App Distribution as the delivery channel and a Google Form as the structured feedback instrument. The application was distributed to 9 invited testers, of whom 7 installed the build and completed the questionnaire — a 78% response rate. While the sample size remains small for statistical generalisation, it is large enough to identify recurring themes in the feedback and allowed the v1.0.0 → v1.0.1 release cycle described below. A larger, more structured evaluation (with formal usability metrics) is identified as future work in Chapter 9.

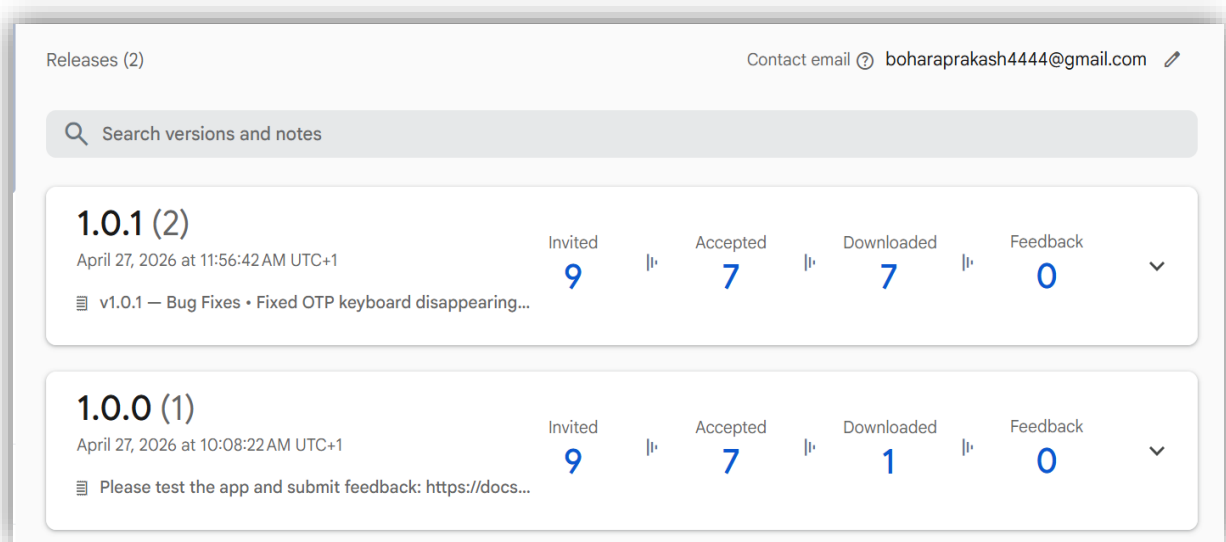


Figure 7.2: Firebase App Distribution dashboard showing release v1.0.1 with 9 invited / 7 accepted / 7 downloaded

Test setup. Each respondent installed the build through App Distribution and completed a structured task list covering the core user journeys: account registration with OTP verification, group creation and member invitation, expense entry through OCR, settlement calculation and payment recording, and analytics review. A 21-question Google Form captured Likert-scale ratings, multiple-choice responses, and open-text feedback. Of nine invited testers, seven completed the form (78% response rate).

Demographics. Respondents spanned four age bands (Under 18 through 26+) with no single band exceeding 28.6%, and reported a range of technology experience (Beginner to Expert), supporting some confidence that findings reflect a varied user profile rather than a homogeneous group.

Quantitative findings. Headline results across the seven respondents are summarised in Table 7.3.

Table 7.3: Headline (User Acceptance Test, n = 7) result

Metric	Result
Overall rating (1–10)	Mean 7.71 (three respondents rated 10, one 9, one 6, one 5, one 4)
Would use the app in real life	71.4% Yes , 14.3% Maybe, 14.3% No
App was easy to use	71.4% agreed
Interface was clear and understandable	71.4% agreed
App responded quickly / did not lag	57.1% / 42.9%
Encountered no major bugs	28.6% (the remainder reported one or more issues, addressed below)

The mean rating of 7.71/10 and the 71.4% real-life-use intent indicate that the pilot group found the application useful and broadly fit for purpose, while leaving meaningful headroom for improvement.

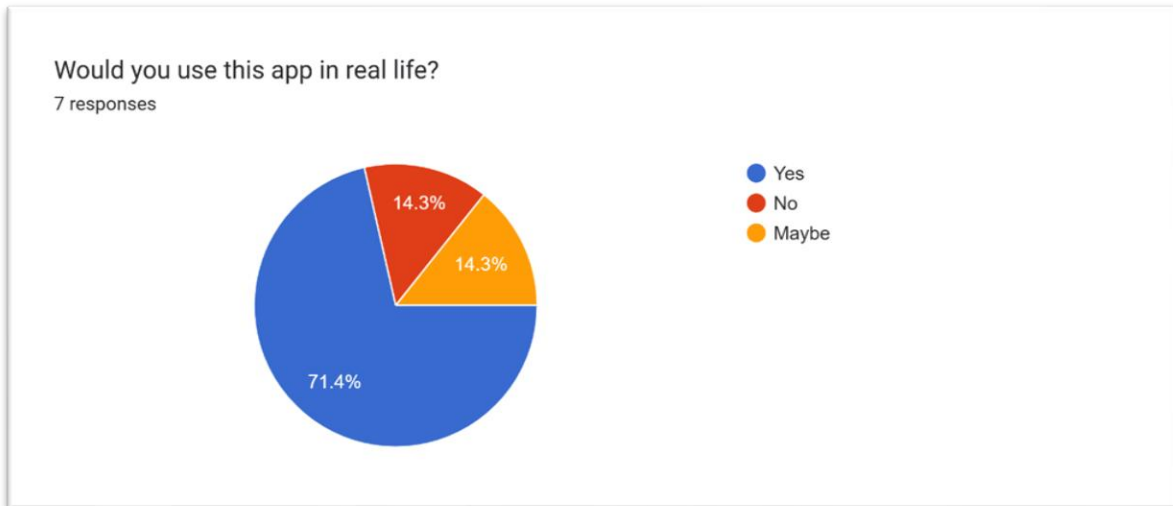


Figure 7.3: Google Form summary — overall rating distribution and would-use-in-real-life chart

Qualitative findings. Open-text responses identified specific strengths and weaknesses of the application. Respondents most consistently valued the receipt-scanning workflow, with one observing that *"the bill scanning feature is very useful because it reduces the need for manual calculations"*. Other recurring strengths were the clean visual design, the clarity of the interface, the security features, and the perceived fairness of group expense management.

The issues raised fell into three categories. **Functional bugs** included the OTP keyboard disappearing during code entry (one respondent; reproducible) and intermittent failures during profile editing and authentication flows (two respondents). **Performance concerns** centred on slow data loading when opening certain screens (multiple respondents). **Feature requests** included non-equal split options (item, percentage, and exact-amount splits, consistent with the FR7 partial-implementation flagged in Chapter 6), a late-payment reminder feature, and explicit biometric activation during onboarding. The functional bugs were addressed in the v1.0.1 release described in Section 7.5; the remaining items are revisited as future work in Chapter 9.

Iterative improvement — v1.0.1 release. Based on the pilot feedback and on issues observed during integration, a v1.0.1 release was distributed via App Distribution.

Table 7.4: records the changes.

Issue	Source	Fix in v1.0.1
OTP keyboard dismissed when user switched to the email app to copy the code	Tester feedback	Keyboard focus retained throughout the OTP entry flow
No pagination on long Expenses and History lists	Tester feedback (data-loading slowness)	"Load more" pagination added to both tabs
Notifications written from the client carried a security risk	Internal review	Notifications now dispatched only via secure Cloud Function (server-side validation + FCM push)
App did not work without a network connection	Tester feedback (offline complaints)	Firestore offline persistence enabled
No automated runtime error reporting	Internal review	Crashlytics wired up for automatic error reporting

This v1.0.0 → v1.0.1 cycle demonstrates the iterative-feedback loop that the Agile methodology described in Chapter 3 is intended to enable: real bugs surfaced by real users, fixed within days, and redistributed to the same testers without manual deployment overhead.

7.6 Strengths, Weaknesses, and Limitations

Strengths. The settlement algorithm is provably optimal under the max-flow min-cut theorem [21], a stronger guarantee than the greedy heuristics used by competing applications. The OTP authentication system applies layered defences (salted hash storage, atomic-transaction rate limiting, constant-time comparison, MX-record validation, account-enumeration resistance, and sessionVersion-based session revocation), implementing the layered authentication model recommended by Bonneau et al. [17]. On-device OCR through ML Kit eliminates network dependency and protects user privacy. Real-time multi-device synchronisation through Firestore listeners gives the application a perceptibly responsive feel without polling overhead. The pilot UAT mean rating of 7.71/10 and the 71.4% real-life-use intent indicate that these technical strengths translate into usable software.

Weaknesses. Three are worth naming directly. First, OCR accuracy is contingent on receipt quality: faded thermal prints, non-standard layouts, and overlapping text degrade extraction reliability. Second, although the implementation supports selective equal-split and pay-on-behalf, percentage and exact split modes are absent from the UI, leaving FR7 only partially met; multiple UAT respondents specifically requested these advanced split modes. Third, automated test coverage is concentrated on the settlement algorithm; widget-level and integration tests remain to be developed.

Limitations of the evaluation. Even with a 78% response rate, the absolute pilot UAT sample of seven respondents is small; the headline numbers should be read as indicative rather than statistically meaningful. The evaluation did not administer the System Usability Scale or measure time-on-task, so usability is reported through respondent self-rating rather than against a standardised benchmark. Cross-platform testing was limited to Android; iOS deployment was deferred from MVP scope and not evaluated.

8 Discussion

This chapter reflects on how the implementation outcomes relate to the aims established in Chapter 1, what the user feedback revealed about the application's real-world fit, and what was learned through the development process.

8.1 Outcomes Against Project Aims

The four project aims were largely met. **Aim 1** (automated bill-splitting through machine-learning receipt scanning) was delivered: ML Kit's on-device text recognition extracts merchant, total, and date from standard supermarket receipts and populates the expense form, eliminating manual entry for the most common case. **Aim 2** (minimising settlement transactions) was delivered with stronger guarantees than originally promised; the implemented MCMF algorithm is provably optimal under the max-flow min-cut theorem, not merely heuristically efficient. **Aim 3** (real-time multi-device collaboration) was delivered through Cloud Firestore listeners, with sub-second propagation observed in functional testing and offline persistence handling reconnection cleanly. **Aim 4** (financial awareness through analytics) was delivered as a category breakdown, monthly trend, and per-member contribution view through the `fl_chart` library.

Two aims were partially rather than fully met. Aim 1's automation is robust on standard receipts but degrades on faded thermal prints and unusual layouts. Aim 4's coverage is broad but not deep; exporting analytics for external analysis (US12) was deferred to future work. Both shortfalls are acknowledged honestly rather than glossed over.

8.2 What the User Feedback Revealed

The pilot UAT (Section 7.5) is the strongest source of external evidence about the application's real-world fit. The headline indicators across seven respondents are encouraging: a mean overall rating of 7.71/10, 71.4% real-life-use intent, and 71.4% endorsement of interface clarity, with three respondents rating the application 10/10. These results

suggest the core value proposition, automated receipt entry combined with a clean and secure interface, resonates with the target user base.

The qualitative feedback identified concrete improvements. The most consistent request was for advanced split modes (percentage and exact-amount splits) beyond the selective equal-split currently implemented; the gap is in the UI surface rather than the underlying engine, validating the FR7 partial-implementation framing in Chapter 6.

Other requests, biometric activation in onboarding, a late-payment reminder feature, and faster data loading, point toward feature priorities for any continuation of the project. Five v1.0.1 fixes (Table 7.4) were shipped mid-evaluation in direct response to this feedback, illustrating that the iterative methodology described in Chapter 3 was applied in practice rather than only on paper.

8.3 Engineering Lessons Learned

Three engineering decisions stand out as genuinely instructive in retrospect.

First, designing the OTP rate limiter as an atomic Firestore transaction on a separate `rate_limits` collection (rather than on the OTP document itself) closed a bypass that would otherwise have allowed an attacker to reset their cooldown counter by deleting their OTP document. The security thinking behind this small piece of code represents the kind of trade-off that production-grade authentication requires.

Second, the `PERMISSION_DENIED` bug discovered during Sprint 4 settlement integration revealed that Firestore subcollection rules do not inherit from the parent document and must be declared explicitly. The lesson is broader than the single bug: cloud-based authorisation is not a one-time setup but an ongoing concern as the schema evolves.

Third, the choice of pence-scaled integer arithmetic for all financial sums prevented an entire class of floating-point rounding bugs that would otherwise have surfaced unpredictably in user-facing balances. The discipline of treating money as integers, and

converting to display-formatted pounds only at the UI boundary, is a small architectural decision with disproportionate downstream value.

9 Conclusion

9.1 Summary

SmartSplit was delivered as an Android Flutter application backed by Firebase, providing automated receipt scanning through on-device Google ML Kit, a provably optimal MCMF settlement algorithm minimising the number of transactions required to clear group debts, real-time multi-device synchronisation with offline persistence, and a multi-dimensional analytics dashboard. User data is protected by a layered security model combining Cloud Functions OTP authentication with hashed storage and atomic-transaction rate limiting, Firestore rules gated by email verification, AES-256-CBC encrypted bank details, and a biometric app-lock. Functional, security, and pilot user-acceptance testing returned a mean rating of 7.71/10 and 71.4% real-life-use intent across seven respondents, indicating that the application is fit for purpose.

9.2 Recommendations and Future Work

Several extensions would meaningfully improve the application or strengthen its evaluation.

1. **Larger, structured UAT with formal usability metrics.** A follow-up evaluation with at least 25–30 respondents, administering the System Usability Scale and time-on-task measurements, would yield statistically meaningful findings rather than the indicative results reported in Chapter 7.
2. **Percentage and exact split UI (FR7 completion).** The data model already supports non-equal splits; only the UI needs implementation. This was the single most-requested improvement in the pilot UAT and is the lowest-effort, highest-value next step. CSV export for external analysis (US12), deferred from the MVP, would similarly address the data-conscious user persona identified in Chapter 4.
3. **iOS deployment and cross-platform validation.** Flutter's design intent is cross-platform, but iOS was deferred from MVP scope. A focused porting and testing

effort would either confirm the cross-platform claim or surface platform-specific issues that the report cannot currently anticipate.

4. **OCR robustness improvements.** The current regex-and-heuristics parser handles standard supermarket receipts well but degrades on faded thermal prints and non-standard layouts. Fine-tuning a lightweight learned model on a corpus of receipt formats would close this gap.
5. **Automated UI and integration test coverage.** Current automated coverage is limited to the settlement algorithm. Adding widget-level tests and end-to-end integration tests would make the manual functional testing reported in Chapter 7 repeatable and catch regressions earlier.
6. **Larger-scale performance and feature testing.** Group sizes above 20 members, expense lists above 1,000 items, and concurrent-write scenarios were not stress-tested. Pilot UAT also surfaced smaller feature requests (late-payment reminders, biometric activation in onboarding, faster screen loading) that would strengthen everyday usability.

9.3 Reflection

The most useful lesson from this project was the value of investing in correctness at the layers that matter most. Designing the settlement algorithm to be provably optimal, rather than merely empirically good, and the authentication layer to apply defence-in-depth across the client and server, both took disproportionate time but provided strong foundations on which the rest of the application could be built confidently. The inverse of this trade-off, broader UI coverage at the expense of algorithmic depth, would have produced a less defensible artefact. That said, the report would have been strengthened by a more structured user-acceptance evaluation administered earlier in the project, and by a broader automated test suite developed alongside features rather than retrofitted; these are the priorities I would adopt at the start of any similar project in future. The combination of building, breaking, fixing, and reflecting that this final-year project required has been the most concentrated software-engineering experience of

my degree, and the resulting application, imperfect but genuinely usable, represents an outcome I am proud of.

10 References

- [1] Statista Research Department, "FinTech – Worldwide," Statista, 2023. [Online]. Available: <https://www.statista.com/outlook/dmo/fintech/worldwide>

- [2] A. Biørn-Hansen, T. A. Majchrzak, and T.-M. Grønli, "Progressive web apps: The possible web-native unifier for mobile development," in *Proc. 13th Int. Conf. Web Information Systems and Technologies (WEBIST)*, Porto, Portugal, 2017, pp. 344–351.

- [3] Flutter Team, "Flutter documentation," Google, 2024. [Online]. Available: <https://docs.flutter.dev>

- [4] Facebook Inc., "React Native — Learn once, write anywhere," 2024. [Online]. Available: <https://reactnative.dev>

- [5] Amazon Web Services, "AWS Amplify documentation," 2024. [Online]. Available: <https://docs.amplify.aws>

- [6] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, May 2015, doi: 10.1038/nature14539.

- [7] R. Smith, "An overview of the Tesseract OCR engine," in *Proc. 9th Int. Conf. Document Analysis and Recognition (ICDAR)*, Curitiba, Brazil, 2007, vol. 2, pp. 629–633, doi: 10.1109/ICDAR.2007.4376991.

- [8] A. Dengel and B. Klein, "smartFIX: A requirements-driven system for document analysis and understanding," in *Proc. 5th IAPR Int. Workshop on Document Analysis Systems (DAS)*, Princeton, NJ, USA, 2002, pp. 433–444.

- [9] Z. Huang, K. Chen, J. He, X. Bai, D. Karatzas, S. Lu, and C. V. Jawahar, "ICDAR2019 competition on scanned receipt OCR and information extraction," in *Proc. Int. Conf. Document Analysis and Recognition (ICDAR)*, Sydney, Australia, 2019, pp. 1516–1520, doi: 10.1109/ICDAR.2019.00244.

- [10] Google ML Kit Team, "ML Kit — On-device machine learning APIs for mobile," Google, 2024. [Online]. Available: <https://developers.google.com/ml-kit>
- [11] Google Cloud, "Vision API — Cloud Vision API documentation," Google, 2024. [Online]. Available: <https://cloud.google.com/vision>
- [12] Firebase Team, "Cloud Firestore documentation," Google, 2024. [Online]. Available: <https://firebase.google.com/docs/firestore>
- [13] Firebase Team, "Cloud Firestore data model," Google, 2024. [Online]. Available: <https://firebase.google.com/docs/firestore/data-model>
- [14] Firebase Team, "Firebase Authentication documentation," Google, 2024. [Online]. Available: <https://firebase.google.com/docs/auth>
- [15] Firebase Team, "Cloud Firestore Security Rules," Google, 2024. [Online]. Available: <https://firebase.google.com/docs/firestore/security/get-started>
- [16] L. Moroney, *The Definitive Guide to Firebase: Build Android Apps on Google's Mobile Platform*, 1st ed. Berkeley, CA, USA: Apress, 2017.
- [17] J. Bonneau, C. Herley, P. C. van Oorschot, and F. Stajano, "The quest to replace passwords: A framework for comparative evaluation of web authentication schemes," in *Proc. IEEE Symp. Security and Privacy*, San Francisco, CA, USA, 2012, pp. 553–567, doi: 10.1109/SP.2012.44.
- [18] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA, USA: MIT Press, 2009.
- [19] T. Verhoeff, "Settling multiple debts efficiently: An invitation to computing science," *Informatics in Education*, vol. 3, no. 1, pp. 105–126, 2004.
- [20] J. Kleinberg and É. Tardos, *Algorithm Design*, 1st ed. Boston, MA, USA: Pearson/Addison-Wesley, 2006.

- [21] A. V. Goldberg and R. E. Tarjan, "Finding minimum-cost circulations by canceling negative cycles," *Journal of the ACM*, vol. 36, no. 4, pp. 873–886, Oct. 1989, doi: 10.1145/76359.76368.
- [22] I. Lee and Y. J. Shin, "Fintech: Ecosystem, business models, investment decisions, and challenges," *Business Horizons*, vol. 61, no. 1, pp. 35–46, Jan./Feb. 2018, doi: 10.1016/j.bushor.2017.09.003.
- [23] P. Gomber, R. J. Kauffman, C. Parker, and B. W. Weber, "On the fintech revolution: Interpreting the forces of innovation, disruption, and transformation in financial services," *Journal of Management Information Systems*, vol. 35, no. 1, pp. 220–265, 2018, doi: 10.1080/07421222.2018.1440766.
- [24] K. Beck, M. Beedle, A. van Bennekum, A. Cockburn, W. Cunningham, M. Fowler, J. Grenning, J. Highsmith, A. Hunt, R. Jeffries, J. Kern, B. Marick, R. C. Martin, S. Mellor, K. Schwaber, J. Sutherland, and D. Thomas, "Manifesto for Agile Software Development," 2001. [Online]. Available: <https://agilemanifesto.org>
- [25] D. A. Norman, *The Design of Everyday Things: Revised and Expanded Edition*. New York, NY, USA: Basic Books, 2013.
- [26] International Organization for Standardization, "ISO 9241-210:2019 — Ergonomics of human-system interaction — Part 210: Human-centred design for interactive systems," ISO, Geneva, Switzerland, 2019.
- [27] D. J. Anderson, *Kanban: Successful Evolutionary Change for Your Technology Business*. Sequim, WA, USA: Blue Hole Press, 2010.
- [28] Splitwise Inc., "Splitwise — Split expenses with friends," 2024. [Online]. Available: <https://www.splitwise.com>
- [29] Settle Up, "Settle Up — Group expenses," 2024. [Online]. Available: <https://settleup.io>

- [30] Tricount, "Tricount — Split group expenses," 2024. [Online]. Available: <https://www.tricount.com>
- [31] Splid, "Splid — Share group expenses," 2024. [Online]. Available: <https://www.splid.app>
- [32] PayPal Holdings Inc., "Venmo," 2024. [Online]. Available: <https://venmo.com>
- [33] Tab, "Tab — The simple way to split a bill," 2024. [Online]. Available: <https://www.tabapp.co>
- [34] GitHub Inc., "GitHub documentation," 2024. [Online]. Available: <https://docs.github.com>
- [35] Microsoft, "Visual Studio Code documentation," Microsoft, 2024. [Online]. Available: <https://code.visualstudio.com/docs>
- [36] Figma Inc., "Figma — Collaborative interface design tool," 2024. [Online]. Available: <https://www.figma.com>
- [37] PlantUML, "PlantUML — Open-source tool for UML diagrams," 2024. [Online]. Available: <https://plantuml.com>
- [38] European Parliament and Council, "Regulation (EU) 2016/679 of the European Parliament and of the Council on the protection of natural persons with regard to the processing of personal data and on the free movement of such data (General Data Protection Regulation)," *Official Journal of the European Union*, vol. L119, pp. 1–88, May 2016.

Appendices

The following appendices provide supporting material referenced throughout the report. Each appendix is self-contained and may be read independently of the main body.

Appendix A: Ethics Approval and Participant Consent

Ethics approval for the user research and pilot user-acceptance testing described in Sections 3.2 and 7.5 was obtained through the University of Roehampton's Computer Science ethics process. The completed ethics form was submitted and approved through the University's online ethics system, and approval has been confirmed by the project supervisor, Kimia Aksir. The original signed form is held within the University's ethics records system.

All UAT participants provided informed consent before downloading the test build through Firebase App Distribution. Personal data collected during testing was limited to the email address required for App Distribution and to Google Form responses; no other identifying information was collected, retained, or shared. Participants were free to withdraw at any time, and respondents who chose not to complete the questionnaire are excluded from the $n=7$ reported in Section 7.5.

Appendix B: User Personas

Full persona cards follow:



Figure A.1: Angel Persona - 1



PERSONA 2 — TRIPTA — THE MARKETING CONSULTANT

Figure B.2: Tripta persons - 2



Appendix C: User Journeys

Three user journeys illustrate how the personas in Section 4.1 interact with SmartSplit's core workflows. Each journey traces an end-to-end task that the named persona would realistically perform.

Journey 1-Angel:

Creating a group and adding the first expense. Angel signs in to SmartSplit and taps **Create Group**, naming it "Flat 23: Monthly Expenses". The app generates a 6-digit invite code, which he shares via WhatsApp. As his three flatmates join, Angel is notified in-app. When he next pays for groceries, he taps **Add Expense** → **Scan Receipt**; ML Kit extracts the total (£47.32) and merchant ("Tesco") on-device (Section 6.5). After confirming the details and selecting **Split Equally**, the expense is saved, all flatmates see it in their notification centre in real time, and balances update to show each owes Angel £11.83. (*Exercises US1, US2, US3, US4, US5, US7.*)

Journey 2-Tripta:

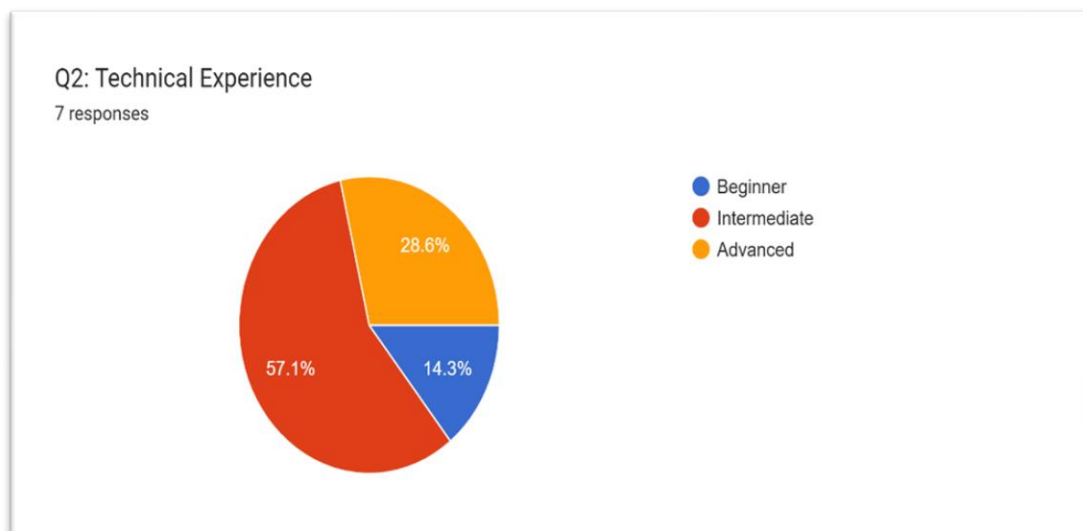
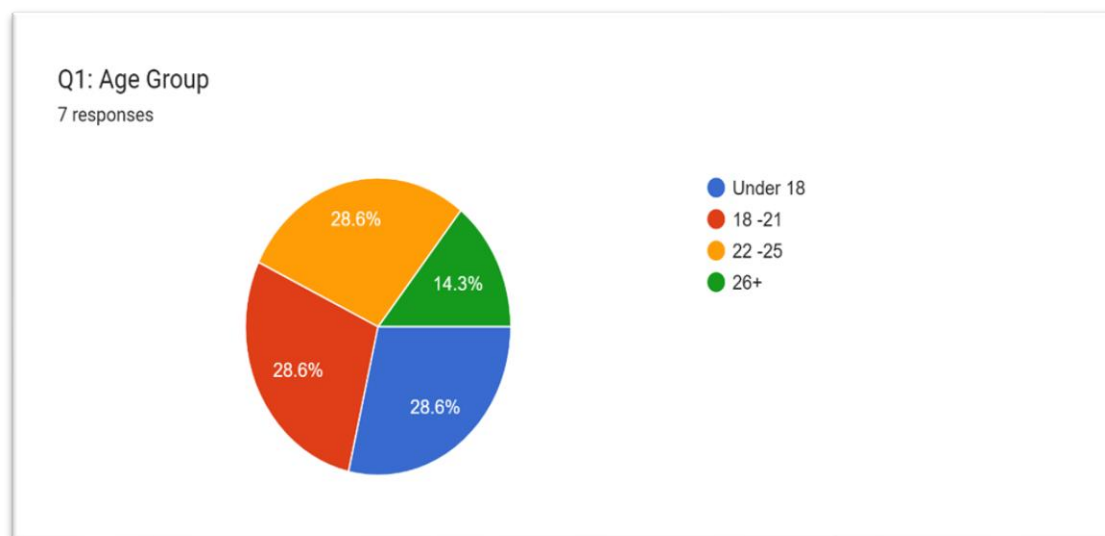
Viewing balances and settling debts. After a weekend trip, Tripta opens the "Weekend in Brighton" group. The **Settlements** screen shows his net position: he owes Ram and Hari, but Kamala owes him. Tapping **Settle Up** reveals the optimised plan computed by the MCMF algorithm (Section 5.6.2), a single payment to Ram that incorporates the offsetting credit from Kamala. Tripta reviews the plan, transfers via bank, marks the settlement as **Paid**, and balances update immediately. Ram is notified that the payment has been recorded. (*Exercises US5, US6, US10.*)

Journey 3- Dikshya:

Reviewing spending analytics. After returning home, Dikshya opens her "Summer Euro Trip" group and navigates to the **Analytics** tab. A pie chart shows the category breakdown (Accommodation 45%, Food 30%, Transport 15%, Activities 10%) and a bar chart shows monthly spending. She filters to the trip's date range and notices spending peaked mid-trip. She shares screenshots with the group to inform planning for the next trip. (*Exercises US8.*)

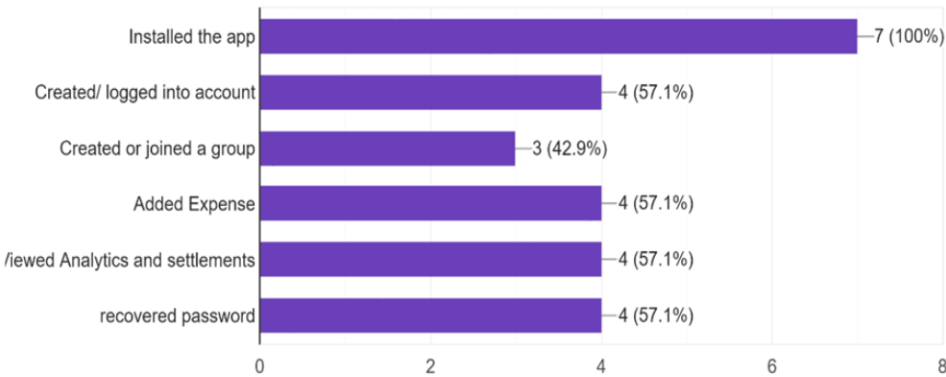
Appendix D: UAT Form and Sample Responses

The form contained 21 questions covering Likert-scale ratings of overall application quality, interface clarity, perceived usefulness of each major feature, and free-text fields for strengths, weaknesses, and improvement suggestions. The seven returned responses (out of nine invited testers) form the quantitative and qualitative basis for Section 7.5.



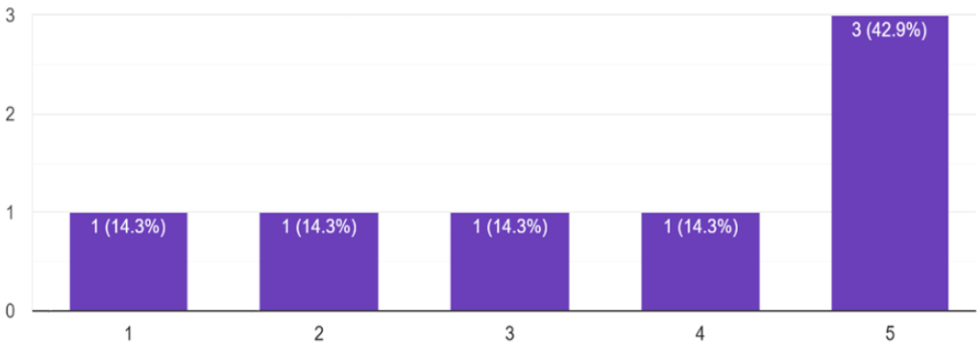
Which tasks did you complete?

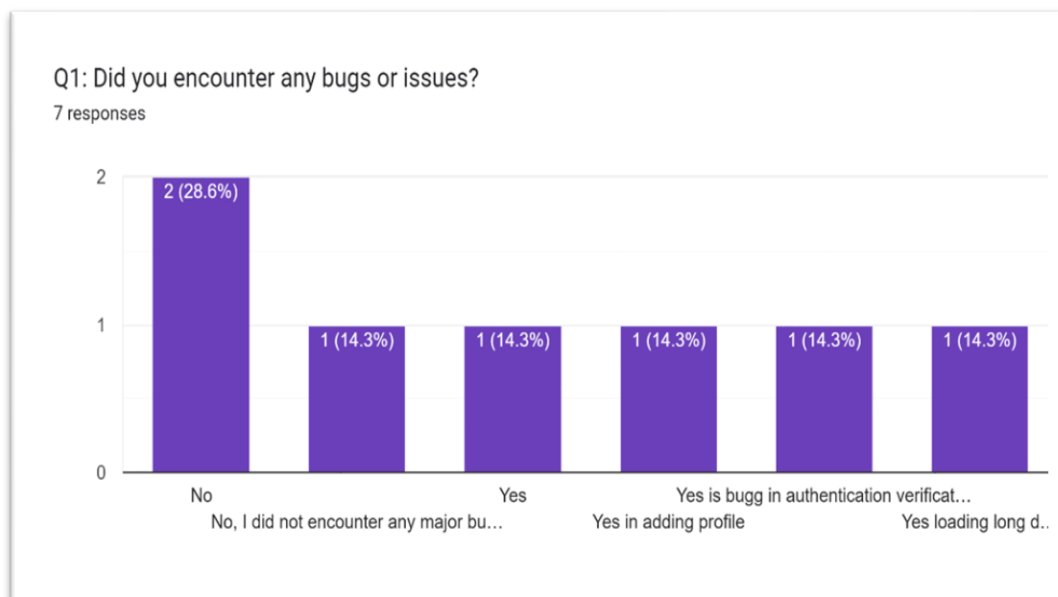
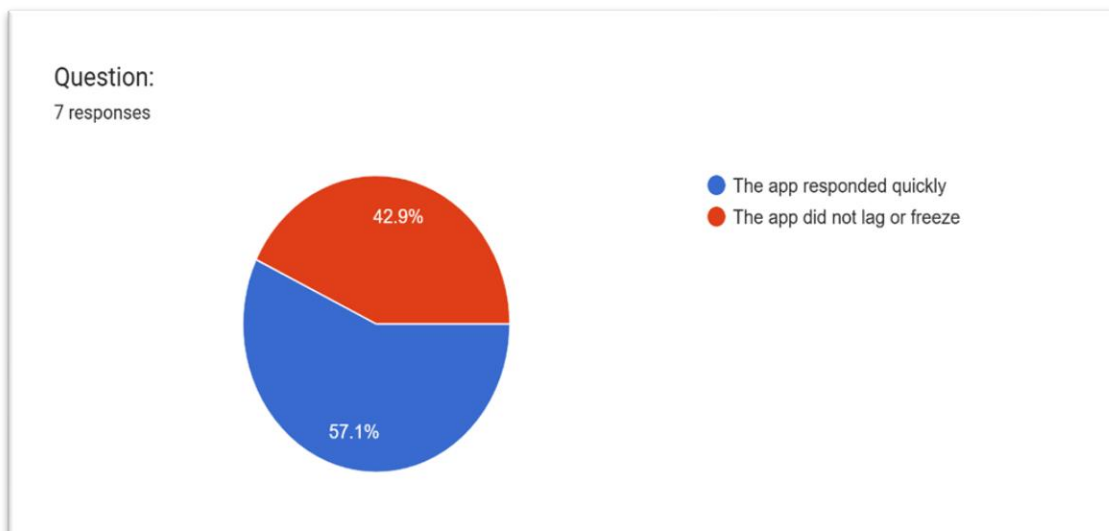
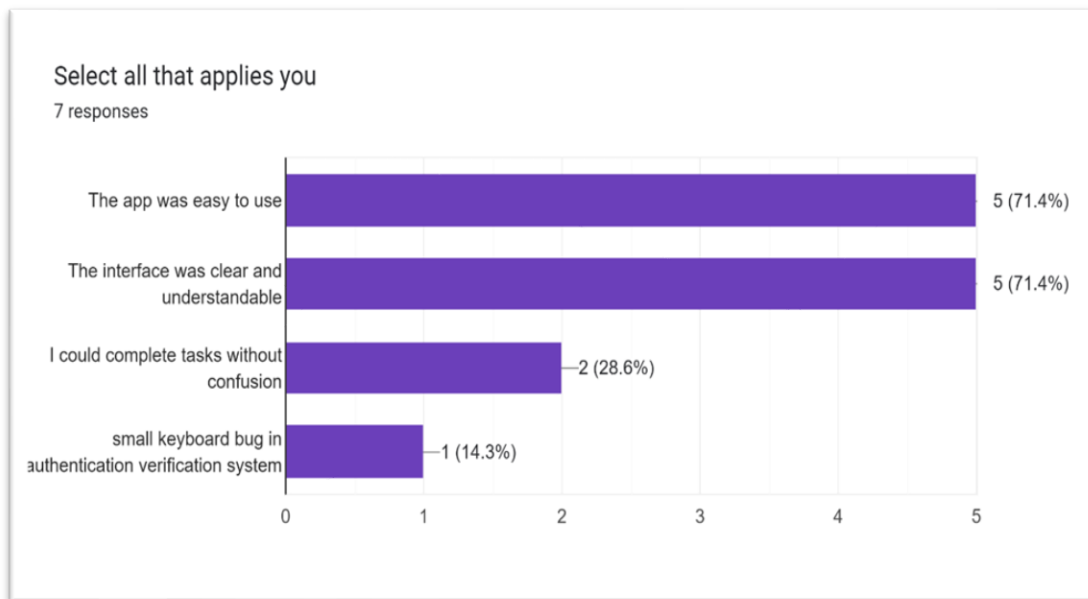
7 responses



Questions:

7 responses





Q2: What did you like about the app?

7 responses

Time management

Security feature reach

Everything

Ui is clean and reach feature

UI design

Features

I really liked how the app makes splitting bills quick, simple, and convenient. The bill scanning feature is very useful because it reduces the need for manual calculations. I also liked that it can help groups manage shared expenses more fairly and efficiently.

Q3: What improvements would you suggest?

7 responses

No

Data loading can needs improvement

If group members do not pay on time, a late charge will be applied to the next payment.

Biometric activation

fix the authentication and offline bug

data fetch

Overall, the app is very useful and well-designed. As a future improvement, it could include more options to customise how the bill is split, such as splitting by item, percentage, or equal share. A payment reminder feature would also make the app even more helpful for group expenses.

Appendix E: Project Proposal Poster

The proposal poster presented at Milestone 1 (November 2025) is available in PDF file below.



poster.pdf